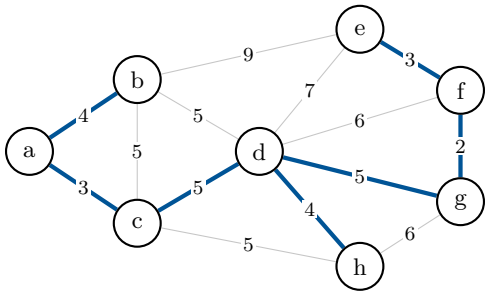
# 作业学习讲义

## 存储 · 遍历 · 最小生成树 · 最短路径




封面图：题 7.2 无向网及其一棵最小生成树（蓝色粗边，总权 26）

**第一部分** 全部原始作业题目（题 7.1 –7.5）

**第二部分** 全部答案与超详细解析（含一题多解、易错点、结果检验）

**第三部分** 题目分类、举一反三、易错专题、知识回顾、科普、科学史、关联学科、高校公开真题、生活应用、学习总结

**第四部分** 参考资料与权威来源

所有计算结果均经程序独立复算核验；所有引用的高校试题均附可访问的公开来源。

编写日期：2026 年 11 月

# 目录

**第一部分　全部原始作业题目** 1

**题 7.1 有向图的度、邻接矩阵与邻接表** 1

**题 7.2 无向网的最小生成树** 1

**题 7.3 由邻接矩阵求 DFS / BFS 生成树** 1

**题 7.4 Dijkstra 单源最短路径** 2

**题 7.5 Floyd 任意两点最短路径** 2

**第二部分　全部答案与超详细解析** 3

**第 7.1 题 有向图的度、邻接矩阵与邻接表** 3

**第 7.2 题 无向网的邻接矩阵、邻接表与最小生成树** 5

**第 7.3 题 由邻接矩阵求深度优先与广度优先生成树** 8

**第 7.4 题 Dijkstra 算法求单源最短路径** 10

**第 7.5 题 Floyd 算法求任意两顶点间最短路径** 12

**第三部分　专题拓展与知识网络** 15

**开篇 题目分类与筛选** 15

**第一章 本次作业知识体系总览** 15

**第二章 重点题深度回顾** 16

2.1 题 7.2：最小生成树 . . . . . 16

2.2 题 7.4：Dijkstra 单源最短路径 . . . . . 16

2.3 题 7.5：Floyd 所有顶点对最短路径 . . . . . 16

**第三章 重点题举一反三** 16

**第四章 易错题专题** 19

4.1 易错 1：有向图的邻接矩阵读法（题 7.1、7.3） . . . . . 19

4.2 易错 2：Prim 与 Dijkstra 混淆（题 7.2、7.4） . . . . . 19

4.3 易错 3：等权边与“结果唯一”的错觉（题 7.2） . . . . . 19

4.4 易错 4：Dijkstra 中“已确定”顶点的处理与负权（题 7.4） . . . . . 19

4.5 易错 5：Floyd 的循环顺序与路径还原（题 7.5） . . . . . 19

4.6 易错 6：遍历顺序与生成树的画法（题 7.3） . . . . . 19

**第五章 易错题举一反三** **20**

**第六章 核心知识系统回顾** **22**

6.1 图的存储：邻接矩阵与邻接表 . . . . . 22
6.2 图的遍历：DFS 与 BFS . . . . . 22
6.3 最小生成树：切分性质与环性质 . . . . . 22
6.4 单源最短路径：松弛与 Dijkstra . . . . . 23
6.5 多源最短路径：Floyd 的动态规划 . . . . . 23
6.6 相似概念辨析 . . . . . 24
6.7 知识迁移：这些知识以后在哪里再出现 . . . . . 24

**第七章 学科科普：图论到底在研究什么** **25**

**第八章 科学史：从七座桥到导航软件** **25**

**第九章 关联学科知识网络与跨课程迁移** **27**

9.1 最小必要知识与小型完整示例 . . . . . 27

**第十章 国内外高校公开历年试题典型题专题** **28**

10.1 跨校横向比较与高频母题 . . . . . 32
10.2 章末考试迁移总结 . . . . . 32

**第十一章 生活中的实际应用** **32**

**第十二章 学习总结** **35**

**第四部分 参考资料与权威来源** **36**

# 第一部分　全部原始作业题目

本部分只收录题目本身。图形均按原题重新绘制（顶点位置、弧的方向与权值与原图一致）。

## 题 7.1

已知如图 1 所示的有向图，请给出该图的：

(1) 每个顶点的入度 / 出度；
(2) 邻接矩阵；
(3) 邻接表。

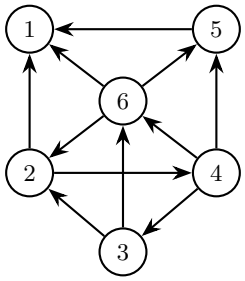



**图 1**　题 7.1 的有向图（原题图重绘）

## 题 7.2

请对如图 2 所示的无向网：

(1) 写出它的邻接矩阵，并按普里姆（Prim）算法求其最小生成树；
(2) 写出它的邻接表，并按克鲁斯卡尔（Kruskal）算法求其最小生成树。

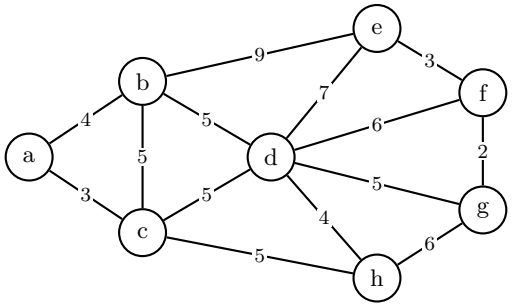



**图 2**　题 7.2 的无向网（原题图重绘）

## 题 7.3

已知二维数组表示的图的邻接矩阵如表 1 所示。试分别画出自顶点 1 出发进行遍历所得的深度优先生成树和广度优先生成树。

**表 1** 题 7.3 的邻接矩阵（第 $i$ 行第 $j$ 列的元素记为 $A[i][j]$）

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 |
| 2 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 3 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 |
| 4 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 5 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 |
| 6 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 7 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| 8 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 |
| 9 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 |
| 10 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |

## 题 7.4

试利用 Dijkstra 算法求如图 3 所示的图中从顶点 a 到其他各顶点间的最短路径，写出执行算法过程中各步的状态。

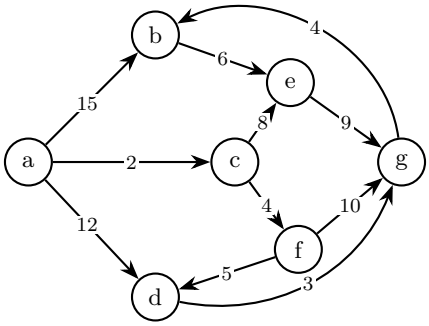



**图 3** 题 7.4 的有向网（原题图重绘；其中 g→b 权 4、d→g 权 3 为两条弧形弧）

## 题 7.5

试利用 Floyd 算法求如图 4 所示的网中任意两顶点之间的最短路径，写出执行算法过程中各步的状态。

| | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | 0 | 2 | 3 | 6 | $\infty$ | $\infty$ |
| 2 | 2 | 0 | $\infty$ | $\infty$ | 4 | 6 |
| 3 | 3 | $\infty$ | 0 | 2 | $\infty$ | $\infty$ |
| 4 | 6 | $\infty$ | 2 | 0 | 1 | 3 |
| 5 | $\infty$ | 4 | $\infty$ | 1 | 0 | $\infty$ |
| 6 | $\infty$ | 6 | $\infty$ | 3 | $\infty$ | 0 |

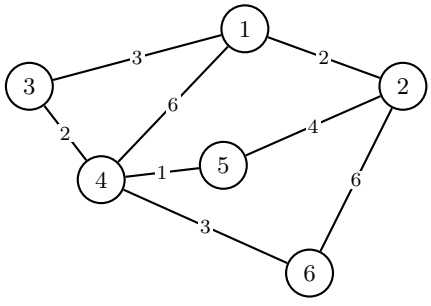



**图 4** 题 7.5 的带权邻接矩阵（左）与无向网（右）（原题重绘）

# 第二部分　全部答案与超详细解析

**统一约定。**顶点按题目给出的编号或字母顺序排列；当算法在若干个候选中“打平”时，统一选择编号（字母）较小者——这是教材和考试最常见的默认约定，换用其他打平规则可能得到**形状不同但同样正确**的结果，文中会逐一指出。所有数值结果均已用 Python 程序独立复算。

## 第 7.1 题 有向图的度、邻接矩阵与邻接表

**①考查知识点**　有向图中顶点的**入度、出度**；**邻接矩阵**与**邻接表**两种存储结构；“度之和 = 弧数”的计数关系。

**②已知条件**　由图 1 读出全部 11 条弧（记号 $\langle u,v\rangle$ 表示从 $u$ 指向 $v$ 的弧，$u$ 称为弧尾，$v$ 称为弧头）：

$$\langle 2,1\rangle,\ \langle 5,1\rangle,\ \langle 6,1\rangle,\ \langle 6,5\rangle,\ \langle 4,5\rangle,\ \langle 6,2\rangle,\ \langle 2,4\rangle,\ \langle 4,6\rangle,\ \langle 3,2\rangle,\ \langle 3,6\rangle,\ \langle 4,3\rangle.$$

**③解题目标**　给出 6 个顶点的入度与出度；写出 $6\times 6$ 邻接矩阵；画出邻接表。

**④解题思路**　所有小问都只依赖同一份“弧清单”。所以第一步是**把图翻译成弧清单**，之后：数“箭头指进来”的弧得入度、数“箭头指出去”的弧得出度；把每条弧 $\langle i,j\rangle$ 在矩阵第 $i$ 行第 $j$ 列记 1 得邻接矩阵；把每个顶点的所有弧头串成一条链得邻接表。最后用“行和 = 出度、列和 = 入度、总和 = 弧数”三重核对。

**定义与公式说明**

**入度** $\mathrm{ID}(v)$：以 $v$ 为弧头的弧的条数；**出度** $\mathrm{OD}(v)$：以 $v$ 为弧尾的弧的条数。
**邻接矩阵**：$n$ 个顶点的有向图用 $n\times n$ 矩阵 $A$ 表示，$A[i][j]=1$ 当且仅当存在弧 $\langle i,j\rangle$，否则为 0。由定义立即得到

$$\mathrm{OD}(i)=\sum_{j=1}^{n}A[i][j]\ (\text{第}i\text{ 行之和}),\qquad \mathrm{ID}(j)=\sum_{i=1}^{n}A[i][j]\ (\text{第}j\text{ 列之和}).$$

**度的计数关系**：每条弧恰好给一个顶点贡献 1 个出度、给一个顶点贡献 1 个入度，故 $\sum_v \mathrm{ID}(v)=\sum_v \mathrm{OD}(v)=e$（$e$ 为弧数）。它对任何有向图成立，本题用作检验。
**邻接表**：为每个顶点设一个表头结点，后接一条单链表，链表中每个结点记录一条以该顶点为弧尾的弧的弧头编号。

**⑤完整推导**

**（1）入度与出度。**逐个顶点检查（解法一：直接数弧）：

- 顶点 1：指入的弧 $\langle 2,1\rangle,\langle 5,1\rangle,\langle 6,1\rangle$，指出的弧无 $\Rightarrow$ ID=3，OD=0；
- 顶点 2：指入 $\langle 6,2\rangle,\langle 3,2\rangle$；指出 $\langle 2,1\rangle,\langle 2,4\rangle$ $\Rightarrow$ ID=2，OD=2；
- 顶点 3：指入 $\langle 4,3\rangle$；指出 $\langle 3,2\rangle,\langle 3,6\rangle$ $\Rightarrow$ ID=1，OD=2；
- 顶点 4：指入 $\langle 2,4\rangle$；指出 $\langle 4,3\rangle,\langle 4,5\rangle,\langle 4,6\rangle$ $\Rightarrow$ ID=1，OD=3；
- 顶点 5：指入 $\langle 6,5\rangle,\langle 4,5\rangle$；指出 $\langle 5,1\rangle$ $\Rightarrow$ ID=2，OD=1；
- 顶点 6：指入 $\langle 4,6\rangle,\langle 3,6\rangle$；指出 $\langle 6,1\rangle,\langle 6,2\rangle,\langle 6,5\rangle$ $\Rightarrow$ ID=2，OD=3。

**（2）邻接矩阵。**把 11 条弧逐条填入（行 = 弧尾，列 = 弧头），并在右侧和下方附上行和、列和：

$$A=$$

| | 1 | 2 | 3 | 4 | 5 | 6 | 行和 (OD) |
|---|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 2 | 1 | 0 | 0 | 1 | 0 | 0 | 2 |
| 3 | 0 | 1 | 0 | 0 | 0 | 1 | 2 |
| 4 | 0 | 0 | 1 | 0 | 1 | 1 | 3 |
| 5 | 1 | 0 | 0 | 0 | 0 | 0 | 1 |
| 6 | 1 | 1 | 0 | 0 | 1 | 0 | 3 |
| 列和 (ID) | 3 | 2 | 1 | 1 | 2 | 2 | 11 |

**解法二（由矩阵求度）**：行和恰为 $(0,2,2,3,1,3)$、列和恰为 $(3,2,1,1,2,2)$，与解法一逐项一致。两种方法相互独立，一致即说明数弧与填矩阵都没有遗漏。

**（3）邻接表。**每个顶点的链表按弧头编号从小到大排列（图 5）。若教材用 0 起始的数组下标存储顶点，则链表结点中的数应统一减 1，结构不变。

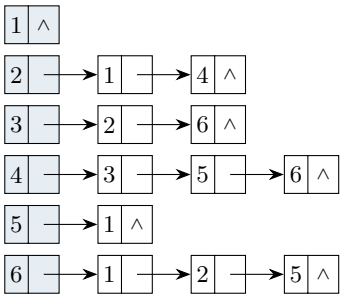



**图 5**　题 7.1 的邻接表。每条链的长度恰好等于该顶点的出度——这就是“邻接表求出度快、求入度慢”的原因

### ⑥最终答案

| 顶点 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 入度 ID | 3 | 2 | 1 | 1 | 2 | 2 |
| 出度 OD | 0 | 2 | 2 | 3 | 1 | 3 |

邻接矩阵见上式 $A$；邻接表见图 5。

**⑦结果检验**　$\sum \mathrm{ID} = 3+2+1+1+2+2 = 11$，$\sum \mathrm{OD} = 0+2+2+3+1+3 = 11$，均等于弧数 11；矩阵中 1 的个数为 11；邻接表链结点总数 $0+2+2+3+1+3 = 11$。三种表示含有同样多的弧，检验通过。

**易错点**

**①把行和当入度。**错误来源：只记住“矩阵和度有关”，没记住方向。正确规则：**行 = 出发点**，所以行和是出度；记忆方法是 $A[i][j]$ 读作“从 $i$ 到 $j$”。**快速自检**：顶点 1 没有任何出弧，第 1 行必须全 0。
**②漏看穿越弧。**弧 $\langle 3,6\rangle$ 从底部竖直穿过横弧 $\langle 2,4\rangle$，两线相交处**不是**顶点，很多同学把它看成两段或漏掉。自检：数完后用 $\sum \mathrm{ID} = \sum \mathrm{OD}$ = 弧数核对。
**③邻接表里写成双向。**无向图的每条边要在两个顶点的链表中各出现一次，有向图的每条弧只在**弧尾**的链表中出现一次。

# 第 7.2 题 无向网的邻接矩阵、邻接表与最小生成树

**①考查知识点**　带权图（网）的邻接矩阵与邻接表；**最小生成树**（MST）的定义；**Prim 算法**与 **Kruskal 算法**的执行过程；MST 的唯一性。

**②已知条件**　8 个顶点 a–h，14 条无向边及权值：

$$(a,b)4,\ (a,c)3,\ (b,c)5,\ (b,d)5,\ (b,e)9,\ (c,d)5,\ (c,h)5,$$
$$(d,e)7,\ (d,f)6,\ (d,g)5,\ (d,h)4,\ (e,f)3,\ (f,g)2,\ (g,h)6.$$

**③解题目标**　写出邻接矩阵、邻接表；分别用 Prim、Kruskal 算法求出 MST（给出过程、边集与总权值）。

**定义：生成树与最小生成树**

连通无向图 $G=(V,E)$ 的**生成树**是包含 $G$ 全部 $n$ 个顶点、且恰有 $n-1$ 条边的连通无环子图。各边权值之和最小的生成树称为**最小生成树**（Minimum Spanning Tree, MST）。本题 $n=8$，所以**任何**生成树都恰有 7 条边。

**定理（MST 切分性质，Prim 与 Kruskal 共同的理论基础）**

设把顶点集任意分成两部分 $U$ 与 $V-U$（称为一个**切分**），若边 $(u,v)$ 是所有“一端在 $U$、另一端在 $V-U$”的边中权值最小的一条，则**一定存在**一棵包含 $(u,v)$ 的最小生成树。
**为什么成立（交换论证）**：任取一棵不含 $(u,v)$ 的 MST $T$，把 $(u,v)$ 加进 $T$ 会形成一个环；这个环从 $U$ 出发又回到 $U$，必然还有另一条跨越切分的边 $(x,y)$。删去 $(x,y)$ 得到的仍是生成树，且权值变化 $w(u,v)-w(x,y)\le 0$，所以新树也是 MST 且包含 $(u,v)$。
**使用条件**：图连通、无向；权值可以为任意实数（包括负数），切分性质照样成立。本题满足。

**④解题思路**　两种算法都是“反复选一条安全的最小边”的贪心法，区别只在**切分怎么取**：

- **Prim**：维护一棵从某个起点（本题取 a）长大的树 $U$，切分固定为 $(U,\ V-U)$，每次选跨越切分的最小边，把新顶点拉进树。需要辅助数组 `closedge`：对每个树外顶点 $v$ 记录“$v$ 到树的最短边”`lowcost[v]` 及其另一端 `adjvex[v]`。
- **Kruskal**：把边按权从小到大排序，依次考察；若一条边的两端当前属于不同连通分量就接受（它是这两个分量之间的最小边，满足切分性质），否则会成环，拒绝。用“并查集”判断是否同属一个分量。

## （1）邻接矩阵与 Prim 算法

**邻接矩阵**（顶点序 a,b,c,d,e,f,g,h；无边记 $\infty$，对角线记 0——也有教材把对角线记为 $\infty$，二者都可以，但全题须一致）：

$$A=\begin{array}{c|cccccccc} & a & b & c & d & e & f & g & h \\ \hline a & 0 & 4 & 3 & \infty & \infty & \infty & \infty & \infty \\ b & 4 & 0 & 5 & 5 & 9 & \infty & \infty & \infty \\ c & 3 & 5 & 0 & 5 & \infty & \infty & \infty & 5 \\ d & \infty & 5 & 5 & 0 & 7 & 6 & 5 & 4 \\ e & \infty & 9 & \infty & 7 & 0 & 3 & \infty & \infty \\ f & \infty & \infty & \infty & 6 & 3 & 0 & 2 & \infty \\ g & \infty & \infty & \infty & 5 & \infty & 2 & 0 & 6 \\ h & \infty & \infty & 5 & 4 & \infty & \infty & 6 & 0 \end{array}$$

无向网的邻接矩阵关于主对角线对称；非 0、非 $\infty$ 元素共 28 个 $=2\times 14$。

**Prim 算法（从 a 出发）。**表 2 中每格写“`lowcost/adjvex`”，“—”表示该顶点已并入树中；每一步在未入树的顶点中选 `lowcost` 最小者（加框），并入后用它到其余树外顶点的边权去**刷新** `lowcost`（只有严格更小才刷新）。

**表 2**　Prim 算法逐步状态（起点 a）

| 步 | 树中顶点集 $U$ | b | c | d | e | f | g | h | 选中边（权） |
|---|---|---|---|---|---|---|---|---|---|
| 0 | $\{a\}$ | 4/a | **3/a** | ∞ | ∞ | ∞ | ∞ | ∞ | (a,c) 3 |
| 1 | $\{a,c\}$ | **4/a** | — | 5/c | ∞ | ∞ | ∞ | 5/c | (a,b) 4 |
| 2 | $\{a,c,b\}$ | — | — | **5/c** | 9/b | ∞ | ∞ | 5/c | (c,d) 5 |
| 3 | $\{a,c,b,d\}$ | — | — | — | 7/d | 6/d | 5/d | **4/d** | (d,h) 4 |
| 4 | $\{a,c,b,d,h\}$ | — | — | — | 7/d | 6/d | **5/d** | — | (d,g) 5 |
| 5 | $\{\ldots,g\}$ | — | — | — | 7/d | **2/g** | — | — | (g,f) 2 |
| 6 | $\{\ldots,g,f\}$ | — | — | — | **3/f** | — | — | — | (f,e) 3 |

逐步说明刷新过程中的关键细节：

- 第 1 步 c 入树：c 的邻边 (c,b)5 不小于 b 当前的 4，不刷新；(c,d)5、(c,h)5 把 d、h 从 ∞ 刷成 5。
- 第 2 步 b 入树：(b,d)5 与 d 当前值 5 相等，**不刷新**（所以 d 的 `adjvex` 仍是 c）；(b,e)9 把 e 刷成 9。此时 d、h 并列最小 5，按约定取字母小的 d。
- 第 3 步 d 入树：(d,e)7<9、(d,f)6、(d,g)5、(d,h)4<5，四处都刷新。
- 第 5 步 g 入树：(g,f)2<6 刷新 f；(g,h) 中 h 已入树，忽略。第 6 步 f 入树：(f,e)3<7 刷新 e。

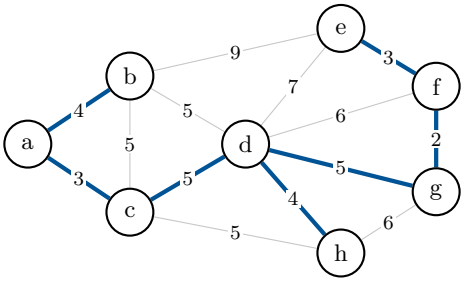



**图 6**　Prim 算法得到的最小生成树（蓝色粗边）。选边次序：(a,c)→(a,b)→(c,d)→(d,h)→(d,g)→(g,f)→(f,e)——树从 a 出发始终连成一片地长大

## （2）邻接表与 Kruskal 算法

**邻接表**：每个链结点含三个域——邻接点、边权、下一结点指针（图 7）。无向网每条边出现两次，链结点共 28 个。

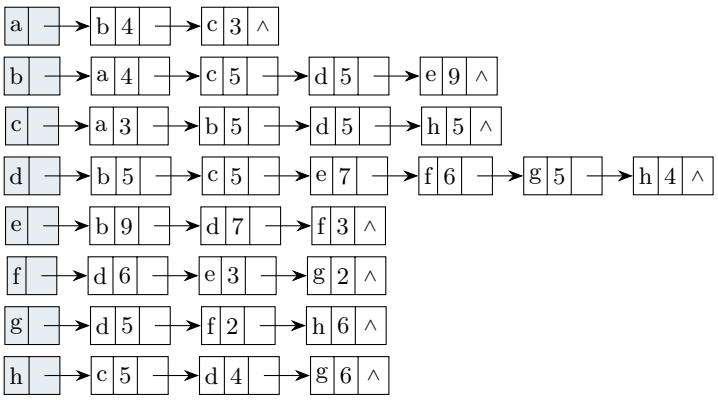



**图 7**　题 7.2 的邻接表（结点格式：邻接点 | 权 | 指针）。链长 = 顶点的度：a2, b4, c4, d6, e3, f3, g3, h3，和为 28

**Kruskal 算法。**把 14 条边按权升序排列（同权按字母序），逐条判断（表 3）。“分量”一栏写出接受该边之后的非单点连通分量。

**表 3**　Kruskal 算法逐步状态

| 序 | 边 | 权 | 判断 | 当前（非单点）连通分量 |
|---|---|---|---|---|
| 1 | (f,g) | 2 | 两端不同分量 ⇒ **接受** | $\{f,g\}$ |
| 2 | (a,c) | 3 | **接受** | $\{a,c\},\{f,g\}$ |
| 3 | (e,f) | 3 | **接受** | $\{a,c\},\{e,f,g\}$ |
| 4 | (a,b) | 4 | **接受** | $\{a,b,c\},\{e,f,g\}$ |
| 5 | (d,h) | 4 | **接受** | $\{a,b,c\},\{d,h\},\{e,f,g\}$ |
| 6 | (b,c) | 5 | b、c 同在 $\{a,b,c\}$，成环 ⇒ 拒绝 | 不变 |
| 7 | (b,d) | 5 | **接受** | $\{a,b,c,d,h\},\{e,f,g\}$ |
| 8 | (c,d) | 5 | 同一分量 ⇒ 拒绝 | 不变 |
| 9 | (c,h) | 5 | 同一分量 ⇒ 拒绝 | 不变 |
| 10 | (d,g) | 5 | **接受**（第 7 条边，结束） | $\{a,b,c,d,e,f,g,h\}$ |

接受满 $n-1=7$ 条边即可停止，(d,f)6、(g,h)6、(d,e)7、(b,e)9 不必再考察（即使考察也都会成环被拒绝）。

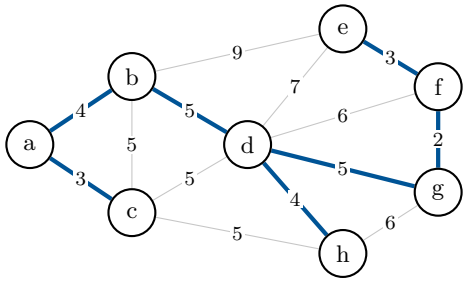



**图 8**　Kruskal 算法得到的最小生成树。选边次序：(f,g)→(a,c)→(e,f)→(a,b)→(d,h)→(b,d)→(d,g)，中间状态是森林。与图 6 相比，(c,d) 换成了 (b,d)，二者权值都是 5

### ⑥最终答案

Prim（起点 a）：边集 $\{(a,c)3,(a,b)4,(c,d)5,(d,h)4,(d,g)5,(g,f)2,(f,e)3\}$，总权 $\boxed{26}$。
Kruskal：边集 $\{(f,g)2,(a,c)3,(e,f)3,(a,b)4,(d,h)4,(b,d)5,(d,g)5\}$，总权 $\boxed{26}$。
两棵树形状不同但都是最小生成树：本图共有**恰好 3 棵** MST，它们都含 (a,b)、(a,c)、(d,g)、(d,h)、(e,f)、(f,g)，再从 {(b,d),(c,d),(c,h)} 中任取一条。

### ⑦结果检验

1. **边数**：两棵树各 7 条边 $=n-1$，且覆盖全部 8 个顶点。
2. **总权**：$3+4+5+4+5+2+3=26$；$2+3+3+4+4+5+5=26$。
3. **最优性（环性质检验）**：对每条非树边 $(u,v)$，它与树上 $u\to v$ 路径构成的环中，它必须是最大边之一。以 Kruskal 树为例：(c,d)5 所成环 c–a–b–d–c 中最大边为 5，满足；(g,h)6 所成环 g–d–h–g 中它是最大边；(b,e)9 所成环 b–d–g–f–e–b 中它最大。逐条检查 7 条非树边均满足，故是 MST。
4. **程序穷举**：枚举全部 $\binom{14}{7}=3432$ 个 7 边子集，其中生成树的最小权确为 26，且恰有 3 个子集达到 26，与上面的唯一性分析一致。

**方法比较**　Prim 以顶点为中心、树始终连通，适合稠密图（用邻接矩阵 $O(n^2)$）；Kruskal 以边为中心、中间状态是森林，适合稀疏图（排序 $O(e\log e)$ 加并查集）。二者本质相同——都是切分性质的两种“切法”；考试中 Prim 常配邻接矩阵、Kruskal 常配边表或邻接表，正是本题两问的设计意图。

**易错点**

**①以为 Prim 与 Kruskal 的结果必须相同。**只有当所有边权互不相同时 MST 才唯一；本题有 5 条权为 5 的边，结果不同完全正常，比较的是总权。
**② Prim 刷新时用“路径长”而不是“边权”。**Prim 的 `lowcost[v]` 是“$v$ 到树的一条边的权”，不是“从起点到 $v$ 的距离”。若写成 `lowcost[v]=lowcost[k]+w(k,v)` 就变成了 Dijkstra（见第三部分第四章）。
**③ Kruskal 只看“两端是否都已出现过”来判环。**正确判据是“两端是否属于同一连通分量”。例如第 7 步考察 (b,d) 时，b 与 d 都已出现在某条已选边上，但分属 $\{a,b,c\}$ 与 $\{d,h\}$，必须接受。

## 第 7.3 题 由邻接矩阵求深度优先与广度优先生成树

**①考查知识点**　从邻接矩阵读出图；**深度优先搜索**（DFS）与**广度优先搜索**（BFS）；遍历生成树（由“首次访问所经过的边”组成）。

**②已知条件**　表 1 的矩阵**不对称**（例如 $A[1][7]=1$ 而 $A[7][1]=0$），因此它表示一个**有向图**，$A[i][j]=1$ 表示弧 $\langle i,j\rangle$。逐行读出各顶点的邻接点（按列号升序）：

$$\begin{array}{llll} 1:\ 7,9 & 2:\ 3,7 & 3:\ 4,8 & 4:\ 5,9 \\ 5:\ 6,10 & 6:\ 1,2 & 7:\ 3,10 & 8:\ 1,4,9 \\ 9:\ 5,7,10 & 10:\ 1,6 & & \end{array}$$

共 22 条弧。图 9 把它画了出来。

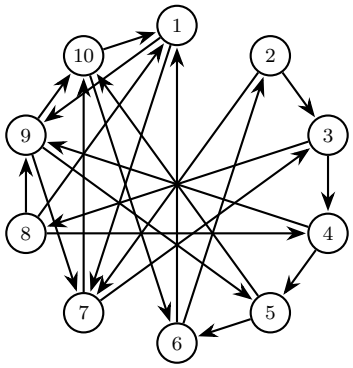



**图 9**　由表 1 读出的有向图（10 个顶点、22 条弧，顶点按圆周排列以减少交叉）

**③解题目标**　从顶点 1 出发分别进行 DFS、BFS（邻接点按编号从小到大访问），画出两棵生成树。

**算法说明**

**DFS**：访问 $v$，然后依次对 $v$ 的每个**未访问**邻接点 $w$ 递归执行 DFS($w$)；“走到底再回溯”，借助（递归）栈实现。
**BFS**：访问 $v$ 并入队；反复取出队头 $u$，把 $u$ 的所有未访问邻接点依次访问并入队；“一层一层向外扩”，借助队列实现。
**遍历生成树**：遍历中，若从 $u$ 出发**首次**访问到 $w$，则弧 $\langle u,w\rangle$ 是树边。连通（或从起点全部可达）时，树边恰有 $n-1$ 条。
**唯一性条件**：结果依赖邻接点的访问顺序。用邻接矩阵存储时，扫描第 $u$ 行的自然顺序就是列号升序，因此本题答案是确定的。

**④解题思路**　DFS 的关键是**记住回溯点**：每走一步都看“当前顶点还有没有没访问过的邻接点”，没有就退回上一个顶点继续看。BFS 的关键是**严格按队列顺序**处理，入队时就标记已访问，避免重复入队。

### （1）深度优先搜索（DFS）

用缩进表示递归深度，“✓”表示邻接点已访问而跳过：

| 递归调用 | 动作 |
|---|---|
| DFS(1) | 访问 1；邻接点 7 未访问 → 树边 $\langle 1,7\rangle$ |
| DFS(7) | 访问 7；邻接点 3 未访问 → $\langle 7,3\rangle$ |
| DFS(3) | 访问 3；邻接点 4 → $\langle 3,4\rangle$ |
| DFS(4) | 访问 4；邻接点 5 → $\langle 4,5\rangle$ |
| DFS(5) | 访问 5；邻接点 6 → $\langle 5,6\rangle$ |
| DFS(6) | 访问 6；1✓；2 → $\langle 6,2\rangle$ |
| DFS(2) | 访问 2；3✓，7✓；返回 |
| （回到 6） | 邻接点处理完，返回 5 |
| （回到 5） | 下一邻接点 10 → $\langle 5,10\rangle$；DFS(10)：访问 10；1✓，6✓；返回 |
| （回到 4） | 下一邻接点 9 → $\langle 4,9\rangle$；DFS(9)：访问 9；5✓，7✓，10✓；返回 |
| （回到 3） | 下一邻接点 8 → $\langle 3,8\rangle$；DFS(8)：访问 8；1✓，4✓，9✓；返回 |
| （回到 7） | 10✓；返回 1；1 的下一邻接点 9✓；结束 |

DFS 访问序列：$1,7,3,4,5,6,2,10,9,8$。

### （2）广度优先搜索（BFS）

| 出队 | 新访问并入队 | 树边 | 队列（出队后、入队后） |
|---|---|---|---|
| — | 1 | — | [1] |
| 1 | 7, 9 | $\langle 1,7\rangle, \langle 1,9\rangle$ | [7, 9] |
| 7 | 3, 10 | $\langle 7,3\rangle, \langle 7,10\rangle$ | [9, 3, 10] |
| 9 | 5（7✓,10✓） | $\langle 9,5\rangle$ | [3, 10, 5] |
| 3 | 4, 8 | $\langle 3,4\rangle, \langle 3,8\rangle$ | [10, 5, 4, 8] |
| 10 | 6（1✓） | $\langle 10,6\rangle$ | [5, 4, 8, 6] |
| 5 | 无（6✓,10✓） | — | [4, 8, 6] |
| 4 | 无（5✓,9✓） | — | [8, 6] |
| 8 | 无 | — | [6] |
| 6 | 2（1✓） | $\langle 6,2\rangle$ | [2] |
| 2 | 无 | — | [ ] |

BFS 访问序列：$1,7,9,3,10,5,4,8,6,2$。

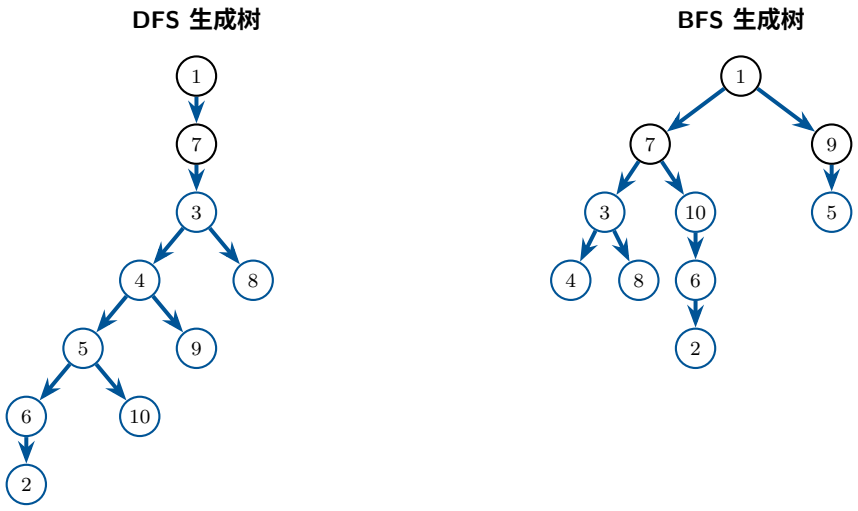



**图 10**　自顶点 1 出发的深度优先生成树（左，深而窄：最长链 1–7–3–4–5–6–2 有 7 个顶点）与广度优先生成树（右，浅而宽：按到 1 的“弧数距离”分为 5 层）

**⑥最终答案**

**DFS 生成树**树弧：$\langle 1,7\rangle, \langle 7,3\rangle, \langle 3,4\rangle, \langle 4,5\rangle, \langle 5,6\rangle, \langle 6,2\rangle, \langle 5,10\rangle, \langle 4,9\rangle, \langle 3,8\rangle$；访问序列 1,7,3,4,5,6,2,10,9,8。
**BFS 生成树**树弧：$\langle 1,7\rangle, \langle 1,9\rangle, \langle 7,3\rangle, \langle 7,10\rangle, \langle 9,5\rangle, \langle 3,4\rangle, \langle 3,8\rangle, \langle 10,6\rangle, \langle 6,2\rangle$；访问序列 1,7,9,3,10,5,4,8,6,2。
两棵树见图 10。

**⑦结果检验**　两棵树都含 10 个顶点、9 条树弧，说明从 1 出发全部可达；每条树弧都能在矩阵中找到对应的 1（如 $\langle 6,2\rangle$ 对应 $A[6][2]=1$）。BFS 树的层次正是各顶点到 1 的最少弧数：例如顶点 2 在第 4 层，而 1 到 2 的确找不到少于 4 条弧的有向路径（2 只能由 6 到达，6 只能由 5 或 10 到达……）。

**易错点**

**①把有向矩阵当作无向图。**很多同学默认“邻接矩阵对称”，于是把第 1 列的 1（来自 6、8、10）也当成 1 的邻接点，DFS 一开始就走向 6，得到 1,6,2,3,…的错误序列。检验方法：先看 $A$ 是否等于 $A^{\mathsf{T}}$。
**② BFS 出队时才标记访问。**这样 10 会被 7 和 9 各放入一次队列，造成重复访问；应在入队时标记。
**③生成树里画进非树弧。**例如 $\langle 9,7\rangle$ 虽存在，但访问 9 时 7 早已被访问，它不是树弧。

## 第 7.4 题 Dijkstra 算法求单源最短路径

**①考查知识点**　带权有向图的单源最短路径；**Dijkstra 算法**的执行过程（集合 $S$、距离数组 `dist`、路径记录）；**松弛**操作。

**②已知条件**　由图 3 读出 11 条弧：

$$\langle a,b\rangle 15,\ \langle a,c\rangle 2,\ \langle a,d\rangle 12,\ \langle b,e\rangle 6,\ \langle c,e\rangle 8,\ \langle c,f\rangle 4,$$
$$\langle e,g\rangle 9,\ \langle f,g\rangle 10,\ \langle f,d\rangle 5,\ \langle d,g\rangle 3,\ \langle g,b\rangle 4.$$

所有权值非负。注意两条容易看漏方向的弧：右上方的弧形弧方向是 $g\to b$（箭头指向 b），下方弧形弧方向是 $d\to g$。

**③解题目标**　求 a 到 b,c,d,e,f,g 的最短路径长度与路径，并写出每一步的状态。

**算法与公式说明**

**松弛**（relaxation）：若已知到 $u$ 的一条路径长 $\mathrm{dist}[u]$，弧 $\langle u,v\rangle$ 权为 $w(u,v)$，则

$$\mathrm{dist}[v]\leftarrow \min\big(\mathrm{dist}[v],\ \mathrm{dist}[u]+w(u,v)\big),$$

意思是“经过 $u$ 再走一步到 $v$，是否比已知的更短”。
**Dijkstra 算法**：$S$ 为已求得最短路径的顶点集，初始 $S=\{a\}$，$\mathrm{dist}[v]=w(a,v)$（无弧为 $\infty$）。重复 $n-1$ 次：在 $V-S$ 中选 dist 最小的顶点 $u$ 加入 $S$，再用 $u$ 松弛它的所有出弧。
**为什么正确（使用条件：所有权值 $\geq 0$）**：设 $u$ 是 $V-S$ 中 dist 最小者。任何到 $u$ 的其他路径都必须在某处第一次离开 $S$ 到达某个 $y\in V-S$，此时长度已 $\geq \mathrm{dist}[y]\geq \mathrm{dist}[u]$，后面的弧权非负只会更长，所以 $\mathrm{dist}[u]$ 已是最短。本题权值都非负，条件满足。

**④解题思路**　把表格想成“候选距离榜”：每一轮把榜上最小的那位“确定下来”（它不可能再被改小），再看它能不能帮别人缩短距离。所以每一轮只做两件事：**选最小、做松弛**。

**⑤逐步执行**　表 4 按严蔚敏教材的经典格式列出每一轮开始时各顶点的 `dist` 与对应路径；方框为该轮选入 $S$ 的顶点，“—”表示已在 $S$ 中。

**表 4**　Dijkstra 算法逐步状态（源点 a）

| 终点 | $i=1$ | $i=2$ | $i=3$ | $i=4$ | $i=5$ | $i=6$ |
|---|---|---|---|---|---|---|
| b | 15 (a,b) | 15 (a,b) | 15 (a,b) | 15 (a,b) | 15 (a,b) | **15 (a,b)** |
| c | **2 (a,c)** | — | — | — | — | — |
| d | 12 (a,d) | 12 (a,d) | **11** (a,c,f,d) | **11 (a,c,f,d)** | — | — |
| e | ∞ | **10** (a,c,e) | **10 (a,c,e)** | — | — | — |
| f | ∞ | **6 (a,c,f)** | — | — | — | — |
| g | ∞ | ∞ | **16** (a,c,f,g) | 16 (a,c,f,g) | **14 (a,c,f,d,g)** | — |
| 选入 $S$ | c {a,c} | f {a,c,f} | e {a,c,f,e} | d {a,c,f,e,d} | g {⋯,d,g} | b 全部 |

逐轮松弛的计算（红色为本轮被改小的值）：

第 1 轮：　初值来自 a 的出弧：b=15，c=2，d=12，其余 ∞。最小为 c=2，c 入 $S$。
第 2 轮：　用 c 松弛：$e$：$\infty \to 2+8=10$；$f$：$\infty \to 2+4=6$。最小为 f=6，f 入 $S$。
第 3 轮：　用 f 松弛：$d$：$12 \to 6+5=11$（更短！）；$g$：$\infty \to 6+10=16$。最小为 e=10，e 入 $S$。
第 4 轮：　用 e 松弛：$g$：$10+9=19>16$，不变。最小为 d=11，d 入 $S$。
第 5 轮：　用 d 松弛：$g$：$16 \to 11+3=14$。最小为 g=14，g 入 $S$。
第 6 轮：　用 g 松弛：$b$：$14+4=18>15$，不变。剩下 b=15，b 入 $S$，算法结束。

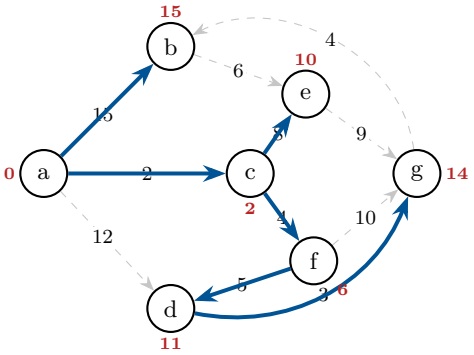



**图 11**　最短路径树（蓝色粗弧）。红字为各顶点的最短距离；每个顶点到 a 的最短路径就是沿树弧反向追溯到 a

### ⑥最终答案

| 终点 | 最短距离 | 最短路径 |
|---|---|---|
| c | 2 | a→c |
| f | 6 | a→c→f |
| e | 10 | a→c→e |
| d | 11 | a→c→f→d |
| g | 14 | a→c→f→d→g |
| b | 15 | a→b |

求得最短路径的先后顺序（即入 $S$ 的顺序）：c, f, e, d, g, b——距离单调不减：
$2 \le 6 \le 10 \le 11 \le 14 \le 15$。

**⑦结果检验**　最短距离必须满足**三角不等式** $\mathrm{dist}[v] \le \mathrm{dist}[u] + w(u,v)$ 对所有 11 条弧成立：$\langle a,b\rangle\ 15 \le 15$，$\langle a,c\rangle\ 2 \le 2$，$\langle a,d\rangle\ 11 \le 12$，$\langle b,e\rangle\ 10 \le 21$，$\langle c,e\rangle\ 10 \le 10$，$\langle c,f\rangle\ 6 \le 6$，$\langle e,g\rangle\ 14 \le 19$，$\langle f,g\rangle\ 14 \le 16$，$\langle f,d\rangle\ 11 \le 11$，$\langle d,g\rangle\ 14 \le 14$，$\langle g,b\rangle\ 15 \le 18$——全部成立，且取等号的弧恰好组成图 11 中的最短路径树。另外用 Bellman–Ford 算法独立复算，结果完全相同。

**易错点**

**①已入 $S$ 的顶点又被“改小”。**Dijkstra 中顶点一旦入 $S$ 就不再更新；正因为权非负，这样做是安全的。本题的回弧 $g \to b$ 给出 $14+4=18$，即使它更小，b 若已在 $S$ 中也不应修改——如果出现这种“需要修改”的情形，说明图中有负权，Dijkstra 本身已不适用（第三部分第五章给出反例）。
**②漏掉“经过新顶点”的更新。**第 3 轮 f 入 $S$ 后，d 从 12 降到 11，这是本题最容易漏掉的一步：a 直接到 d 的弧并不是最短。
**③看错弧形弧方向。**若把 $d \to g$ 误读为 $g \to d$，g 的距离会变成 16，整个答案出错。读图时先列弧清单，并标明方向。

# 第 7.5 题 Floyd 算法求任意两顶点间最短路径

**①考查知识点**　多源（所有顶点对）最短路径；**Floyd 算法**的动态规划思想；距离矩阵序列 $D^{(k)}$ 与路径矩阵 $P^{(k)}$；路径还原。

**②已知条件**　6 个顶点的无向网，8 条边：$(1,2)2$，$(1,3)3$，$(1,4)6$，$(2,5)4$，$(2,6)6$，$(3,4)2$，$(4,5)1$，$(4,6)3$，带权邻接矩阵见图 4。

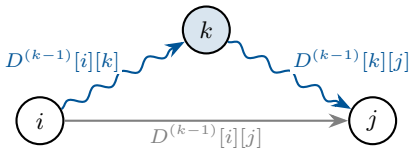
**③解题目标**　求全部顶点对之间的最短路径长度和路径，写出每一轮的状态。

**Floyd 算法的递推公式**

记 $D^{(k)}[i][j]$ 为“从 $i$ 到 $j$、中间只允许经过顶点 $1,2,\dots,k$ 的最短路径长度”。$D^{(0)}$ 就是带权邻接矩阵（不经过任何中间点）。递推式为

$$D^{(k)}[i][j]=\min\Big(D^{(k-1)}[i][j],\ \ D^{(k-1)}[i][k]+D^{(k-1)}[k][j]\Big),\qquad k=1,2,\dots,n.$$

**含义**：允许多用一个中间顶点 $k$ 时，最短路径要么根本不经过 $k$（第一项），要么经过 $k$ 恰好一次，被 $k$ 分成两段，每段都只用到 $1,\dots,k-1$（第二项）。$D^{(n)}$ 即为所求。
**使用条件**：允许负权边，但不能有负权回路；本题权值全为正，满足。
**路径矩阵**：本讲义用 $P[i][j]$ 表示“当前 $i\to j$ 最短路径上 $j$ 的**前驱**顶点”。初值：有边则 $P[i][j]=i$，否则记“-”。当经过 $k$ 更新 $D[i][j]$ 时，令 $P[i][j]\leftarrow P[k][j]$。还原路径时从 $j$ 出发不断取前驱直到 $i$。




两条候选取较小者：
灰：不经过 $k$；蓝：经过 $k$
（两段内部只用顶点 $1..k-1$）

**图 12**　Floyd 算法的核心：每加入一个“允许中转”的顶点 $k$，就检查所有 $(i,j)$ 是否值得绕道 $k$

**④解题思路**　一共做 $n=6$ 轮，第 $k$ 轮把顶点 $k$ 当作“中转站”。一个省力技巧：第 $k$ 轮中，**第 $k$ 行和第 $k$ 列不会改变**（因为 $D[k][k]=0$），只需用第 $k$ 行、第 $k$ 列的“十字”去更新其他位置；无向图矩阵对称，只算上三角再对称抄写即可。

**⑤逐轮状态**　下面每个矩阵中，蓝色为本轮“十字”（第 $k$ 行、第 $k$ 列），**红色粗体**为本轮被改小的元素；右侧 $P^{(k)}$ 为路径（前驱）矩阵，红色为本轮改动的位置。

$$D^{(0)}=\begin{bmatrix}0&2&3&6&\infty&\infty\\2&0&\infty&\infty&4&6\\3&\infty&0&2&\infty&\infty\\6&\infty&2&0&1&3\\\infty&4&\infty&1&0&\infty\\\infty&6&\infty&3&\infty&0\end{bmatrix}\qquad P^{(0)}=\begin{bmatrix}-&1&1&1&-&-\\2&-&-&-&2&2\\3&-&-&3&-&-\\4&-&4&-&4&4\\-&5&-&5&-&-\\-&6&-&6&-&-\end{bmatrix}$$

**第 1 轮（$k=1$，允许经过顶点 1）**：更新（只列 $i<j$，对称位置同步更新）：$d_{23}:\infty\to 2+3=5$；$d_{24}:\infty\to 2+6=8$。

$$D^{(1)}=\begin{bmatrix}0&2&3&6&\infty&\infty\\2&0&\mathbf{5}&\mathbf{8}&4&6\\3&\mathbf{5}&0&2&\infty&\infty\\6&\mathbf{8}&2&0&1&3\\\infty&4&\infty&1&0&\infty\\\infty&6&\infty&3&\infty&0\end{bmatrix}\qquad P^{(1)}=\begin{bmatrix}-&1&1&1&-&-\\2&-&\mathbf{1}&\mathbf{1}&2&2\\3&\mathbf{1}&-&3&-&-\\4&\mathbf{1}&4&-&4&4\\-&5&-&5&-&-\\-&6&-&6&-&-\end{bmatrix}$$

**第 2 轮（$k=2$，允许经过顶点 1,2）**：更新（只列 $i<j$，对称位置同步更新）：$d_{15}:\infty\to 2+4=6$；$d_{16}:\infty\to 2+6=8$；$d_{35}:\infty\to 5+4=9$；$d_{36}:\infty\to 5+6=11$；$d_{56}:\infty\to 4+6=10$。

$$D^{(2)}=\begin{bmatrix}0&2&3&6&\mathbf{6}&\mathbf{8}\\2&0&5&8&4&6\\3&5&0&2&\mathbf{9}&\mathbf{11}\\6&8&2&0&1&3\\\mathbf{6}&4&\mathbf{9}&1&0&\mathbf{10}\\\mathbf{8}&6&\mathbf{11}&3&\mathbf{10}&0\end{bmatrix}\qquad P^{(2)}=\begin{bmatrix}-&1&1&1&\mathbf{2}&\mathbf{2}\\2&-&1&1&2&2\\3&1&-&3&\mathbf{2}&\mathbf{2}\\4&1&4&-&4&4\\\mathbf{2}&5&\mathbf{1}&5&-&\mathbf{2}\\\mathbf{2}&6&\mathbf{1}&6&\mathbf{2}&-\end{bmatrix}$$

**第 3 轮（$k=3$，允许经过顶点 1,2,3）**：更新（只列 $i<j$，对称位置同步更新）：$d_{14}:6\to 3+2=5$；$d_{24}:8\to 5+2=7$。

$$D^{(3)}=\begin{bmatrix}0&2&3&\mathbf{5}&6&8\\2&0&5&\mathbf{7}&4&6\\3&5&0&2&9&11\\\mathbf{5}&\mathbf{7}&2&0&1&3\\6&4&9&1&0&10\\8&6&11&3&10&0\end{bmatrix}\qquad P^{(3)}=\begin{bmatrix}-&1&1&\mathbf{3}&2&2\\2&-&1&\mathbf{3}&2&2\\3&1&-&3&2&2\\\mathbf{3}&\mathbf{1}&4&-&4&4\\2&5&1&5&-&2\\2&6&1&6&2&-\end{bmatrix}$$

**第 4 轮（$k=4$，允许经过顶点 1,2,3,4）**：更新（只列 $i<j$，对称位置同步更新）：$d_{35}:9\to 2+1=3$；$d_{36}:11\to 2+3=5$；$d_{56}:10\to 1+3=4$。

$$D^{(4)}=\begin{bmatrix}0&2&3&5&6&8\\2&0&5&7&4&6\\3&5&0&2&\mathbf{3}&\mathbf{5}\\5&7&2&0&1&3\\6&4&\mathbf{3}&1&0&\mathbf{4}\\8&6&\mathbf{5}&3&\mathbf{4}&0\end{bmatrix}\qquad P^{(4)}=\begin{bmatrix}-&1&1&3&2&2\\2&-&1&3&2&2\\3&1&-&3&\mathbf{4}&\mathbf{4}\\3&1&4&-&4&4\\2&5&\mathbf{4}&5&-&\mathbf{4}\\2&6&\mathbf{4}&6&\mathbf{4}&-\end{bmatrix}$$

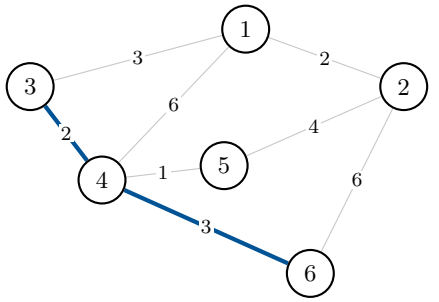
**第 5 轮（$k=5$，允许经过顶点 1,2,3,4,5）**：更新（只列 $i<j$，对称位置同步更新）：$d_{24}:7\to 4+1=5$。

$$D^{(5)}=\begin{bmatrix}0&2&3&5&6&8\\2&0&5&\mathbf{5}&4&6\\3&5&0&2&3&5\\5&\mathbf{5}&2&0&1&3\\6&4&3&1&0&4\\8&6&5&3&4&0\end{bmatrix}\qquad P^{(5)}=\begin{bmatrix}-&1&1&3&2&2\\2&-&1&\mathbf{5}&2&2\\3&1&-&3&4&4\\3&\mathbf{5}&4&-&4&4\\2&5&4&5&-&4\\2&6&4&6&4&-\end{bmatrix}$$

**第 6 轮（$k=6$，允许经过顶点 1,2,3,4,5,6）**：没有任何元素被更新（$D^{(6)}=D^{(5)}$）。

$$D^{(6)}=\begin{bmatrix}0&2&3&5&6&8\\2&0&5&5&4&6\\3&5&0&2&3&5\\5&5&2&0&1&3\\6&4&3&1&0&4\\8&6&5&3&4&0\end{bmatrix}\qquad P^{(6)}=\begin{bmatrix}-&1&1&3&2&2\\2&-&1&5&2&2\\3&1&-&3&4&4\\3&5&4&-&4&4\\2&5&4&5&-&4\\2&6&4&6&4&-\end{bmatrix}$$

最后一轮 $k=6$ 没有更新：顶点 6 只与 2、4 相连且离其他顶点都较远，任何路径绕道 6 都不会更短。




例：还原 3→6
$P[3][6]=4\Rightarrow 6$ 的前驱为 4
$P[3][4]=3\Rightarrow 4$ 的前驱为 3
路径 3→4→6，长度 2+3=5

**图 13**　利用前驱矩阵 $P^{(6)}$ 还原最短路径（以 3→6 为例）

## ⑥最终答案

$$D^{(6)}=\begin{bmatrix}0&2&3&5&6&8\\2&0&5&5&4&6\\3&5&0&2&3&5\\5&5&2&0&1&3\\6&4&3&1&0&4\\8&6&5&3&4&0\end{bmatrix}$$

| 对 | 路径 (长度) | 对 | 路径 (长度) | 对 | 路径 (长度) |
|---|---|---|---|---|---|
| 1–2 | 1–2 (2) | 2–3 | 2–1–3 (5) | 3–5 | 3–4–5 (3) |
| 1–3 | 1–3 (3) | 2–4 | 2–5–4 (5) | 3–6 | 3–4–6 (5) |
| 1–4 | 1–3–4 (5) | 2–5 | 2–5 (4) | 4–5 | 4–5 (1) |
| 1–5 | 1–2–5 (6)* | 2–6 | 2–6 (6) | 4–6 | 4–6 (3) |
| 1–6 | 1–2–6 (8)* | 3–4 | 3–4 (2) | 5–6 | 5–4–6 (4) |

* 有两条等长最短路径：1–5 还可走 1–3–4–5，1–6 还可走 1–3–4–6，长度相同。

## ⑦结果检验

1. **对称性**：无向图的 $D^{(6)}$ 必须对称，满足。
2. **三角不等式**：对任意 $i,j,k$ 都有 $D[i][j]\le D[i][k]+D[k][j]$，程序逐一检查 216 组全部满足。
3. **解法二（逐点 Dijkstra）**：以每个顶点为源点各运行一次 Dijkstra（共 6 次），得到的 6 行距离与 $D^{(6)}$ 完全一致。
4. **单调性**：每轮元素只减不增，$\infty$ 最终全部变为有限值，说明图连通。

**方法比较**　Floyd 一次算出全部 $n^2$ 个距离，代码只有三重循环，时间 $O(n^3)$，适合稠密小图、并能处理负权边；逐点 Dijkstra 用堆实现时总时间约 $O(n\,e\log n)$，在稀疏大图上更快，但要求权非负。考试手算时 Floyd 的“十字法”格式统一、不易漏项。

**易错点**

**①把 $k$ 放在最内层循环。**正确顺序是 `for k / for i / for j`。若写成 `for i / for j / for k`，计算 $D[i][j]$ 时用到的 $D[i][k]$、$D[k][j]$ 可能还没被算好，结果可能错误（第三部分第五章给出 4 个顶点的反例）。

**②用本轮刚改过的值能否继续用？**可以。第 $k$ 轮中第 $k$ 行、第 $k$ 列本身不变，所以“原地更新”与严格使用 $D^{(k-1)}$ 结果相同——这也是 Floyd 只需一个矩阵的原因。

**③路径还原时把 $P$ 的含义弄反。**不同教材的 $P$ 含义不同（前驱、后继或“中间点”），答题时须先写明约定，再按约定还原。

# 第三部分　专题拓展与知识网络

## 开篇 题目分类与筛选

综合知识重要度、典型度、考试频率、方法代表性、综合性、易错度与可迁移度，把五道题分为四类（同一题可属多类）：

**表 5**　本次作业题目分类

| 类别 | 题号 | 入选理由 |
|---|---|---|
| **A 重点核心题** | 7.2、7.4、7.5 | 最小生成树与两类最短路径是“图的应用”的三大母题，几乎是所有数据结构期末、考研 408 与国外 Algorithms 课程的必考内容。 |
| **B 典型基础题** | 7.1、7.3 | 存储结构与遍历是后续一切图算法的“地基”：MST、最短路径都要在矩阵或邻接表上扫描邻接点。 |
| **C 易错题** | 7.3、7.4、7.2 | 7.3 的矩阵不对称（有向）；7.4 有回弧 g→b 与“绕路更短”的 d；7.2 有 5 条等权边，结果不唯一。 |
| **D 综合题** | 7.5、7.2 | 7.5 融合动态规划、矩阵运算与路径还原；7.2 同时考两种存储与两种算法。 |

## 第一章　本次作业知识体系总览

这五道题放在一起，本质上在训练一件事：**把一个“关系网”存进计算机，然后在上面系统地“走”——要么走遍（遍历），要么用最少的代价连通（最小生成树），要么用最短的路线到达（最短路径）**。图 14 把它画成了知识地图。

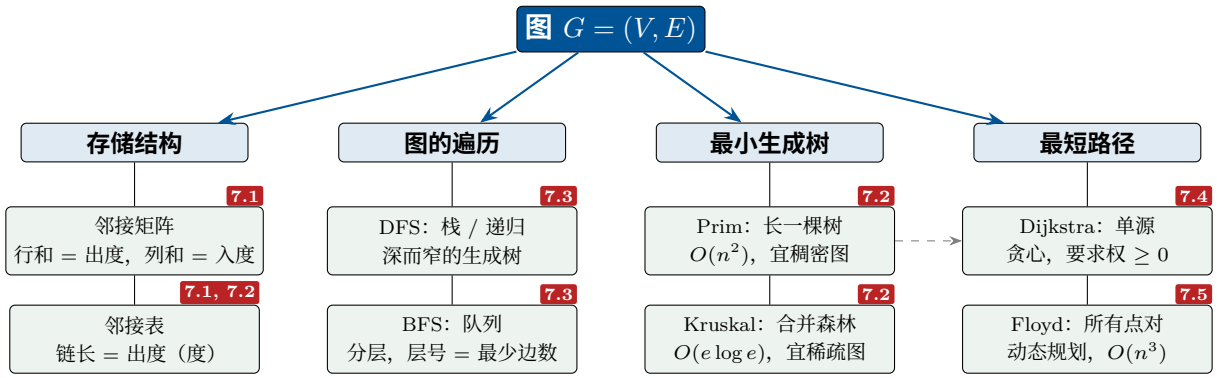



**图 14**　第七章知识地图。右上角红色标签为对应的作业题号；虚线提示 Prim 与 Dijkstra 的代码骨架几乎相同，只是“比较的量”不同

**表 6**　知识点—核心概念—公式/定理—题号—用途

| 知识点 | 核心概念 | 典型公式 / 定理 | 题号 | 常见用途 |
|---|---|---|---|---|
| 存储结构 | 邻接矩阵、邻接表 | $\mathrm{OD}(i)=\sum_j A[i][j]$；$\sum\mathrm{ID}=\sum\mathrm{OD}=e$ | 7.1, 7.2 | 稠密图用矩阵、稀疏图用表 |
| 图的遍历 | DFS、BFS、遍历生成树 | 树边数 $=n-1$；BFS 层号 = 最少边数 | 7.3 | 连通性、可达性、无权最短路 |
| 最小生成树 | 生成树、切分、安全边 | 切分性质、环性质 | 7.2 | 管网 / 线路 / 聚类 |
| 单源最短路 | 松弛、集合 $S$ | $d[v]\leftarrow\min(d[v],d[u]+w)$；权 $\geq 0$ 时贪心正确 | 7.4 | 导航、路由 |
| 多源最短路 | 中转点、DP 状态 | $D^{(k)}=\min(D_{ij}^{(k-1)},D_{ik}^{(k-1)}+D_{kj}^{(k-1)})$ | 7.5 | 距离表、传递闭包 |

# 第二章 重点题深度回顾

## 2.1 题 7.2：最小生成树

**为什么重要。**MST 是“用最少代价把所有点连起来”这一工程问题的标准模型，同时是学习**贪心算法正确性证明**（交换论证）的第一个完整例子；后续《算法设计与分析》中的拟阵、并查集都从这里出发。
**训练的能力。**按规则执行算法并保持状态表；识别等权边带来的不唯一；用切分 / 环性质**验证**一个答案是否最优，而不仅是“算出来”。
**母题结构。**给定连通无向网 $G$，求 $\min_T \sum_{e\in T} w(e)$，$T$ 取遍生成树。换数字、换图、换成“城市铺光缆”“村庄修路”的文字题，只要“连通全部点 + 总代价最小 + 无方向”，都是同一题。
**识别信号。**“连通所有……”“总造价最低”“铺设”“架设”；无起点终点要求；边无方向。

## 2.2 题 7.4：Dijkstra 单源最短路径

**为什么重要。**它是一切导航、路由的核心原型，也是“贪心 + 正确性依赖条件（非负权）”的典范，考试中常配“写出各步状态”或“给出求得最短路径的顶点顺序”。
**训练的能力。**维护 $S$ 与 dist 的不变式；识别“绕路反而更短”（本题 a→d 直达 12，绕道 c、f 只要 11）；用三角不等式检验结果。
**母题结构。**给定源点 $s$、非负权有向（或无向）网，求 $\delta(s,v)$ 对所有 $v$。变化方式：改权值、改源点、改成单终点（反向图）、加“必经点”（拆成两段）。
**识别信号。**“从某点出发到其余各点”“最短时间 / 最省费用”；权值非负；需要具体路线。

## 2.3 题 7.5：Floyd 所有顶点对最短路径

**为什么重要。**它是本科阶段第一个“以矩阵为状态”的动态规划算法；同一框架换一种运算就能求传递闭包（Warshall）、最大容量路径等，迁移性极强。
**训练的能力。**理解 DP 状态“只允许经过 $1..k$”；按“十字法”系统更新矩阵；用前驱矩阵还原路径。
**母题结构。**$n$ 阶带权矩阵 $W$ → 距离矩阵 $D$ 与路径矩阵 $P$；变化方式：有向图、加边 / 删边后的增量更新、求“图的中心”（使最远距离最小的顶点）。
**识别信号。**“任意两顶点之间”“每对城市”“距离表”；顶点数不大；可能含负权但无负回路。

# 第三章 重点题举一反三

每道变式只改变**一个**关键条件，并给出完整解答。所有答案已用程序（含穷举）核验。

**变式 7.2-1（基础迁移）：把边 (c,h) 的权由 5 改为 3**

**题目。**其余条件同题 7.2，求最小生成树及其总权，并判断是否唯一。
**考查点。**Kruskal 执行；MST 唯一性判定。
**思路。**改一条边的权只会影响它在排序中的位置，按 Kruskal 重新走一遍即可。
**推导。**排序：(f,g)2，(a,c)3，(c,h)3，(e,f)3，(a,b)4，(d,h)4，(b,c)5，(b,d)5，(c,d)5，(d,g)5，⋯
接受 (f,g)、(a,c)、(c,h)、(e,f)、(a,b)、(d,h)，此时分量为 $\{a,b,c,d,h\}$ 与 $\{e,f,g\}$。(b,c)、(b,d)、(c,d) 两端都在同一分量，拒绝；(d,g) 连接两个分量，接受，共 7 条边。
**答案。**$T=\{(f,g)2,(a,c)3,(c,h)3,(e,f)3,(a,b)4,(d,h)4,(d,g)5\}$，总权 $\boxed{24}$，**唯一**。
**检验（环性质）。**每条非树边在其环中都是**严格**最大：(b,c)5 > 环中 4,3；(b,d)5 > 环 b–a–c–h–d 中最大 4；(c,d)5 > 3,4；(d,f)6 > 5,2；(g,h)6 > 5,4；(d,e)7 > 5,2,3；(b,e)9 > 环中最大 5。严格大于 ⇒ 不能交换 ⇒ 唯一。
**与原题比较。**原题中 (b,d)、(c,d)、(c,h) 三条 5 都可以接通 $\{a,b,c\}$ 与 $\{d,h\}$，造成 3 棵 MST；把 (c,h) 降为 3 后它提前把 h 接入，等权竞争消失。

**变式 7.2-2（条件变化）：规定生成树必须包含边 (d,e)**

**题目。**在题 7.2 的网中，求包含边 (d,e) 的最小权生成树。

**思路。**方法一：先把 (d,e) 放入，再对其余边执行 Kruskal（“预先合并”）。方法二：在原 MST 上加入 (d,e) 形成唯一的环，删去环上除 (d,e) 外权最大的边。

**推导（方法一）。**先选 (d,e)；再依次接受 (f,g)2、(a,c)3、(e,f)3（$\{d,e,f,g\}$）、(a,b)4、(d,h)4（$\{d,e,f,g,h\}$）；(b,c) 拒绝；(b,d)5 连通 $\{a,b,c\}$ 与其余部分，接受，共 7 条。

**推导（方法二）。**在 Kruskal 树中加入 (d,e)7，形成环 d–e–f–g–d，环上其余边为 (e,f)3、(f,g)2、(g,d)5，删最大的 (g,d)5：$26+7-5=28$。

**答案。**最小权为 $\boxed{28}$，一棵为 $\{(d,e),(f,g),(a,c),(e,f),(a,b),(d,h)\}$（(b,d) 换成 (c,d) 或 (c,h) 同样为 28）。

**为什么原方法仍可用。**强制选边相当于把 d、e “预先缩成一个点”，在缩点后的图上切分性质照样成立。

**变式 7.2-3（综合提升）：MST 有几棵？哪些边“必选 / 必不选”？**

**题目。**对题 7.2 的网，判断：(1) MST 共有几棵；(2) 哪些边属于所有 MST；(3) 哪些边不属于任何 MST。

**思路。**按权值分层考察（Kruskal 的思想）：同一权值层内，看哪些边在连接不同分量、有没有“多选一”。

**推导。**权 $\le 4$ 的 5 条边 (f,g)、(a,c)、(e,f)、(a,b)、(d,h) 互不成环，必须全选，得到分量 $A=\{a,b,c\}$、$B=\{d,h\}$、$C=\{e,f,g\}$。权 5 的边中：(b,c) 在 $A$ 内部，必不选；连接 $A$ 与 $B$ 的有 (b,d)、(c,d)、(c,h) 三条，只需其一；连接 $B$ 与 $C$ 的只有 (d,g)，必选。此后已连通，权 6、7、9 的边全部成环。

**答案。**(1) $\boxed{3}$ 棵；(2) 必选：(a,b)、(a,c)、(d,g)、(d,h)、(e,f)、(f,g)；(3) 必不选：(b,c)、(d,f)、(g,h)、(d,e)、(b,e)。

**检验。**程序枚举全部 $\binom{14}{7}=3432$ 种 7 边组合，权为 26 的生成树恰有 3 棵，与推理一致。

**变式 7.4-1（基础迁移）：把弧 $\langle a,c\rangle$ 的权由 2 改为 9**

**推导。**初值 b=15，c=9，d=12。

第 1 轮选 c=9，松弛：e=17，f=13。第 2 轮选 d=12，松弛：g=15。第 3 轮选 f=13，松弛：d 已在 $S$；g：$13+10=23>15$ 不变。第 4 轮 b、g 并列 15，按约定选 b，松弛：e：$15+6=21>17$ 不变。第 5 轮选 g=15：b 已在 $S$。第 6 轮选 e=17。

**答案。**b 15 (a,b)；c 9 (a,c)；d 12 (a,d)；e 17 (a,c,e)；f 13 (a,c,f)；g 15 (a,d,g)。

**与原题比较。**原题“绕道 c、f 到 d”更短，是因为 a→c 很便宜；c 变贵后直达弧 a→d 反而最优——最短路径树的形状对单条边权很敏感，但算法步骤完全不变。

**变式 7.4-2（条件变化）：求其他各顶点到 g 的最短路径**

**思路。**“多源到单终点”等价于在**反向图**（每条弧掉头）上做“单源”。反向图中 g 的出弧为 $g\to d$ 3、$g\to e$ 9、$g\to f$ 10 等。

**推导。**在反向图上从 g 出发：初值 d=3，e=9，f=10。选 d=3：松弛 a=15（原弧 a→d 12），f=$3+5=8$（原弧 f→d）。选 f=8：c=$8+4=12$。选 e=9：b=$9+6=15$。选 c=12：a=$12+2=14<15$。选 a=14，最后选 b=15。

**答案。**d→g 3；f→d→g 8；e→g 9；c→f→d→g 12；a→c→f→d→g 14；b→e→g 15。

**检验。**a 到 g 的距离 14 与原题从 a 出发求得的 $\mathrm{dist}[g]=14$ 一致——同一条路径从两个方向算，必须相等。

**变式 7.4-3（综合提升）：求从 a 到 g 且必须经过 e 的最短路径**

**思路。**必经点把问题拆成两段独立的最短路：$\delta(a,e)+\delta(e,g)$。前一段已由原题得到 10；e 的出弧只有 $e\to g$ 9，故 $\delta(e,g)=9$。

**答案。**$10+9=\boxed{19}$，路径 a→c→e→g。比不加限制的 14 多 5，这是“必须经过 e”付出的代价。

**注意。**两段拼接可能重复经过同一顶点；在非负权图中若出现重复，去掉中间的环不会变长，因此“拆

两段”的结果仍是正确最小值。

**变式 7.5-1（基础迁移）：把边 (4,5) 的权由 1 改为 5**

**推导。**按同样的 6 轮 Floyd 计算（程序复算）得

$$D=\begin{bmatrix}0&2&3&5&6&8\\2&0&5&\mathbf{7}&4&6\\3&5&0&2&\mathbf{7}&5\\5&\mathbf{7}&2&0&\mathbf{5}&3\\6&4&\mathbf{7}&\mathbf{5}&0&\mathbf{8}\\8&6&5&3&\mathbf{8}&0\end{bmatrix}$$

红色为与原题不同的元素：$d_{24}$ 5→7（原来走 2–5–4，现在 2–1–3–4），$d_{35}$ 3→7（3–1–2–5 为 9，3–4–5 为 7），$d_{45}$ 1→5，$d_{56}$ 4→8（5–4–6 为 8，5–2–6 为 10）。

**启示。**边 (4,5) 原来是“交通枢纽”：它出现在 2–4、3–5、5–6 等多条最短路径上，一旦变贵，影响面就很广。

**变式 7.5-2（条件变化）：删去边 (1,4)，哪些距离会变？**

**思路。**只有当某对顶点的**所有**最短路径都用到 (1,4) 时，删去它才会让距离变大。

**推导。**(1,4) 权 6，而 1–3–4 只要 5，严格更短，所以任何经过 (1,4) 的路径都能用 1–3–4 替换得到更短的路径——(1,4) 不在任何最短路径上。

**答案。**距离矩阵**完全不变**（程序验证）。这类边称为“冗余边”。

**变式 7.5-3（综合提升）：新增一条边 (3,6)，权 1，不重新跑 Floyd 如何更新？**

**思路。**新边只可能被“用一次”（正权图中最短路不会重复经过同一条边）。所以新的距离只有三种可能：不用新边、按 3→6 方向用、按 6→3 方向用：

$$D'[i][j]=\min\big(D[i][j],\ D[i][3]+1+D[6][j],\ D[i][6]+1+D[3][j]\big).$$

**推导。**逐对计算，只有两对变化：$D'[1][6]=\min(8,\ D[1][3]+1+D[6][6])=\min(8,3+1+0)=4$；$D'[3][6]=\min(5,0+1+0)=1$。其余如 $D'[4][6]=\min(3,\ 2+1+0)=3$ 不变。

**答案。**仅 $d_{16}$：8→4，$d_{36}$：5→1（及对称位置）。用这一增量公式只需 $O(n^2)$ 次运算，而重新运行 Floyd 需 $O(n^3)$。程序对加边后的整图重跑 Floyd，结果一致。

# 第四章 易错题专题

难题不等于易错题。下面六个“陷阱”有的计算极其简单，却是作业和考试中失分最多的地方。

## 4.1 易错 1：有向图的邻接矩阵读法（题 7.1、7.3）

**错误来源。**概念不清 + 思维定势：先学的无向图矩阵是对称的，于是默认所有邻接矩阵都对称。
**错误示范。**把题 7.3 当无向图：顶点 1 的邻接点取为第 1 行和第 1 列中所有 1 的位置——6,7,8,9,10，于是 DFS 为 1,6,2,3,4,5,9,7,10,8。
**为什么错。**$A[i][j]=1$ 只说明“$i$ 能到 $j$”，不说明“$j$ 能到 $i$”。第 1 列的 1 来自 6、8、10 指向 1 的弧，从 1 出发沿这些弧是逆行。
**正确规则。**先检查 $A=A^{\mathsf{T}}$ 是否成立；不对称就是有向图。DFS/BFS 只扫描第 $u$ 行。
**快速自检。**树弧 $\langle u,w\rangle$ 必须满足 $A[u][w]=1$（行在前、列在后）。

## 4.2 易错 2：Prim 与 Dijkstra 混淆（题 7.2、7.4）

**错误来源。**两者代码骨架几乎相同（都是“选最小—更新”），区别只有一行。
**错误示范。**在 Prim 中写 `lowcost[v] = lowcost[k] + w(k,v)`，或在 Dijkstra 中写 `dist[v] = w(k,v)`。
**为什么错。**Prim 比较的是“$v$ 连到整棵树的一条边多长”，Dijkstra 比较的是“$v$ 离源点的整条路径多长”。目标不同，得到的树也不同（第五章变式 E-3 用题 7.2 的图算出两者相差 11）。
**判断规则。**题目问“总代价最小地连通所有点”⇒ Prim/Kruskal；问“从某点到各点最短”⇒ Dijkstra。

## 4.3 易错 3：等权边与“结果唯一”的错觉（题 7.2）

**错误示范。**“我用 Prim 得到 (c,d)，标准答案是 (b,d)，所以我错了。”
**为什么错。**MST 唯一的充分条件是“边权互不相同”；本题不满足。判分看的是：过程是否符合算法规则、总权是否最小。
**正确处理。**答题时写明打平规则（如“同权取字母序小者”），并给出总权。
**快速自检。**环性质：每条非树边在它与树形成的环中，必须是最大边之一。

## 4.4 易错 4：Dijkstra 中“已确定”顶点的处理与负权（题 7.4）

**错误示范 A。**第 3 轮 f 入 $S$ 后忘记用 f 更新 d，最终给出 d=12（直达弧）。
**错误示范 B。**看到回弧 g→b 就想“再回头修改 b”。
**为什么错。**A 违反了“每个新入 $S$ 的顶点都要松弛其全部出弧”；B 混淆了 Dijkstra 与 Bellman–Ford——Dijkstra 正确性的前提是非负权，在此前提下已入 $S$ 的顶点不可能被改小，因此根本不需要回头。
**快速自检。**入 $S$ 的距离序列必须单调不减（本题 2,6,10,11,14,15）；若你的表中出现“后入 $S$ 的距离反而更小”，一定是漏了松弛。

## 4.5 易错 5：Floyd 的循环顺序与路径还原（题 7.5）

**错误示范。**`for i: for j: for k: D[i][j]=min(D[i][j], D[i][k]+D[k][j])`。
**为什么错。**DP 的阶段变量是“允许的中转点集合 $\{1..k\}$”，必须在最外层逐步扩大；把 $k$ 放进最内层，计算 $D[i][j]$ 时所需的 $D[k][j]$（$k>i$）还没算好。
**正确规则。**$k$ 最外层；第 $k$ 轮先写出“十字”，再更新其余元素。
**快速自检。**最终矩阵必须满足三角不等式；无向图必须对称。

## 4.6 易错 6：遍历顺序与生成树的画法（题 7.3）

**错误示范。**BFS 出队时才标记已访问，导致 10 被入队两次；或在生成树中画进 $\langle 9,7\rangle$ 这样的非树弧。
**正确规则。**入队即标记；只有“首次发现”的弧才是树弧，树弧数必须为 $n-1$。
**快速自检。**BFS 树中任意树弧连接的两顶点层号恰好相差 1。

# 第五章　易错题举一反三

**变式 E-1（针对易错 1）：由题 7.3 的矩阵直接求各顶点入度、出度**

**最易犯的错误。**按“对称矩阵”思维把行和当作“度”。

**正确解法。**出度 = 行和，入度 = 列和：

| 顶点 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 和 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 出度（行和） | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 3 | 3 | 2 | 22 |
| 入度（列和） | 3 | 1 | 2 | 2 | 2 | 2 | 3 | 1 | 3 | 3 | 22 |

**检验。**两行之和都等于矩阵中 1 的个数 22。

**防错原则。**$A[i][j]$ 永远读作“从 $i$ 到 $j$”：行看出发、列看到达。

**变式 E-2（针对易错 4）：负权弧让 Dijkstra 失效**

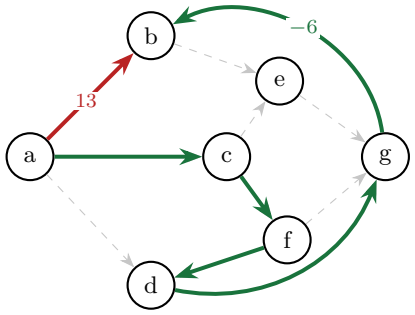
**题目。**在题 7.4 的图中把 $\langle a,b\rangle$ 改为 13，把 $\langle g,b\rangle$ 改为 $-6$（其余不变）。用 Dijkstra 求 a 到 b 的“最短距离”，并与真实值比较。

**执行。**入 $S$ 顺序为 c(2)、f(6)、e(10)、d(11)、**b(13)**、g(14)。b 在 g 之前就被确定为 13；g 入 $S$ 时虽然发现 $14+(-6)=8<13$，但 b 已在 $S$ 中，算法不再修改。

**真实值。**路径 a→c→f→d→g→b 长 $2+4+5+3-6=\boxed{8}$（Bellman–Ford 复算确认；图中回路 b→e→g→b 权 $6+9-6=9>0$，无负权回路，最短路有定义）。

**错误原因。**Dijkstra 的正确性证明用到了“后面的弧权非负，路径只会越走越长”；负权弧打破了这一点。

**防错原则。**有负权弧 ⇒ 用 Bellman–Ford（或无负回路时的 Floyd）；只有**全部**权非负才用 Dijkstra。




红：Dijkstra 确定的 b=13
绿：真实最短路 8
b 先于 g 出队，来不及修正

**图 15**　负权弧使 Dijkstra 出错的反例（变式 E-2）

**变式 E-3（针对易错 2）：同一张图上的 MST 与最短路径树**

**题目。**在题 7.2 的无向网中，以 a 为源点分别求 Prim 最小生成树与 Dijkstra 最短路径树，比较两棵树的总权与“a 到 f 的树上距离”。

**解答。**Prim 树（第二部分已求）总权 26，树上 a 到 f 的路径 a–c–d–g–f 长 $3+5+5+2=15$。

Dijkstra 从 a 出发：dist = b4, c3, d8 (a,c,d), h8 (a,c,h), e13 (a,b,e), g13 (a,c,d,g), f14 (a,c,d,f)。最短路径树的边为 (a,b),(a,c),(c,d),(c,h),(b,e),(d,g),(d,f)，总权 $4+3+5+5+9+5+6=\boxed{37}$，但 a 到 f 只要 14。

**结论。**MST 让“总长”最小（$26<37$），最短路径树让“从源点出发的每条路”最短（$14<15$）。两个目标不同，所以不能互相替代。

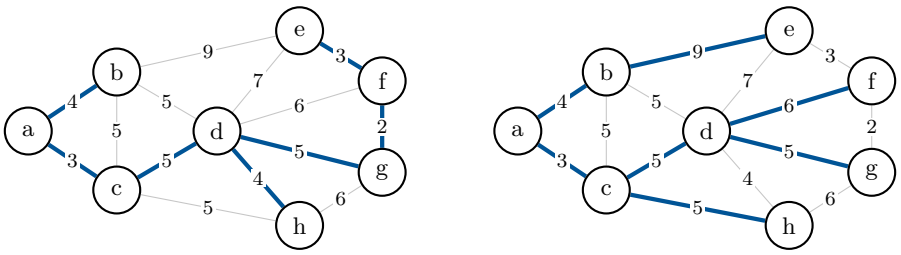



**图 16**　同一张网上的 MST（左）与最短路径树（右）。右图中 (b,e)9、(d,f)6 是为了“让 e、f 离 a 更近”而选的，在 MST 中它们都不划算

**变式 E-4（针对易错 5）：Floyd 循环顺序写错的反例**

**题目。**有向图 4 个顶点，弧 $\langle 1,2\rangle, \langle 2,1\rangle, \langle 2,4\rangle, \langle 4,3\rangle$，权均为 1。分别用正确顺序（$k,i,j$）和错误顺序（$i,j,k$）计算 $D[1][3]$。

**正确。**路径 1→2→4→3，$D[1][3]=3$。

**错误顺序的执行。**计算 $(i,j)=(1,3)$ 时依次试 $k=1..4$：$D[1][2]+D[2][3]=1+\infty$（$D[2][3]$ 要到 $i=2$ 时才算）；$D[1][4]+D[4][3]=\infty+1$（$D[1][4]$ 在 $j=4$ 时才算，晚于 $j=3$）。于是 $D[1][3]$ 停留在 $\boxed{\infty}$，错误。

**防错原则。**DP 必须先算“小阶段”再算“大阶段”；Floyd 的阶段是 $k$，所以 $k$ 在最外层。

**变式 E-5（针对易错 6）：用“头插法”建邻接表后再遍历**

**题目。**若按行读入题 7.3 的矩阵、并用**头插法**建立邻接表（后读到的邻接点插在链表最前面），从 1 出发的 DFS、BFS 序列是什么?

**最易犯的错误。**以为“同一张图，遍历结果一定相同”。

**解答。**头插法使每条链表变为列号**降序**：1: 9,7；2: 7,3；3: 8,4；4: 9,5；5: 10,6；6: 2,1；7: 10,3；8: 9,4,1；9: 10,7,5；10: 6,1。

DFS：1,9,10,6,2,7,3,8,4,5；BFS：1,9,7,10,5,3,6,8,4,2。

**检验。**两个序列都覆盖 10 个顶点；与第二部分的结果不同但都正确。

**防错原则。**遍历序列 = 图 + 存储结构 + 邻接点顺序。题目说“画出遍历所得的生成树”时，若存储方式已给定（本题为矩阵），顺序就是唯一的。

# 第六章　核心知识系统回顾

## 6.1　图的存储：邻接矩阵与邻接表

**是什么。**邻接矩阵用 $n \times n$ 数组记录“谁和谁相邻”；邻接表为每个顶点挂一条链，只存真正存在的边。
**解决什么问题。**让算法能快速回答两类查询：“$u, v$ 之间有没有边?”（矩阵 $O(1)$）与“$u$ 的所有邻居是谁?”（邻接表 $O(\deg u)$）。
**为什么这样设计。**矩阵空间恒为 $\Theta(n^2)$，与边数无关；邻接表空间为 $\Theta(n+e)$。当 $e \ll n^2$（稀疏图，例如道路网）时邻接表省得多。
**最易混淆。**有向 / 无向的对称性；邻接表求入度要扫描全部链表（或另建逆邻接表）。

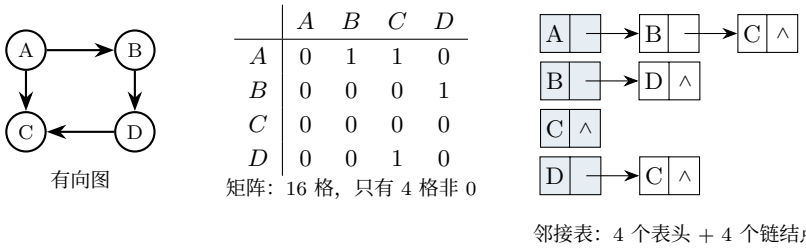



**图 17**　同一个有向图的两种存储。顶点越多、边越“稀”，邻接表的空间优势越明显

## 6.2　图的遍历：DFS 与 BFS

**是什么。**系统地访问图中每个顶点恰好一次。DFS 一路走到底再回溯，BFS 先访问近邻再访问远邻。
**核心性质。**BFS 访问顶点的层号等于它到起点的**最少边数**——所以 BFS 就是“所有边权为 1”时的最短路径算法；DFS 的回溯结构可用于判环、拓扑排序和求强连通分量。
**复杂度。**邻接表 $O(n+e)$，邻接矩阵 $O(n^2)$。
**最易混淆。**DFS 用栈（递归）、BFS 用队列；遍历序列依赖邻接点顺序。

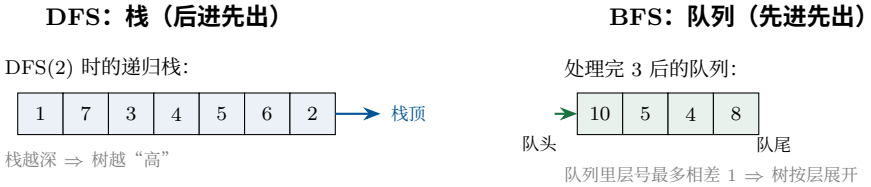



**图 18**　题 7.3 中两种遍历的“工作区”快照：DFS 的栈记录了一条从根出发的路径；BFS 的队列中只含相邻两层的顶点

## 6.3　最小生成树：切分性质与环性质

**是什么。**连通无向网中边权和最小的生成树。
**核心定理。**切分性质：跨越任意切分的最小边可放心选入；环性质：任意环上的严格最大边一定不在 MST 中。Prim 用固定形式的切分 $(U, V-U)$，Kruskal 用“当前两分量之间”的切分。
**为什么成立。**第二部分已用交换论证给出；图 19 是它在题 7.2 上的具体样子。
**使用条件。**无向、连通；对有向图的对应问题叫“最小树形图”，需要完全不同的算法（朱—刘算法 / Edmonds 算法，属拓展内容）。

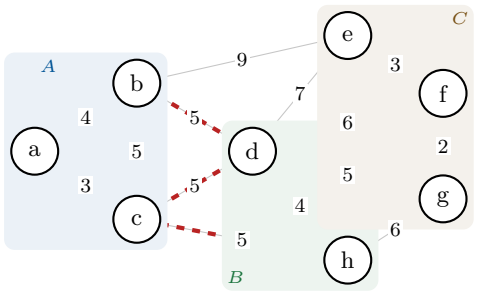



**图 19** 切分性质在题 7.2 上的样子：MST 的“不唯一”恰好来自同一切分上有多条并列最小边

## 6.4 单源最短路径：松弛与 Dijkstra

**是什么。**给定源点，求到其余各点的最短路径。
**核心操作。**松弛 $d[v] \leftarrow \min(d[v], d[u] + w(u, v))$；所有最短路径算法（Dijkstra、Bellman–Ford、DAG 最短路）都只是“以不同顺序做松弛”。
**为什么贪心可行。**非负权保证“$S$ 外距离最小的顶点已无法被改进”（证明见第二部分）。
**复杂度。**数组实现 $O(n^2)$（适合稠密图，也是教材手算的版本）；二叉堆实现 $O((n + e)\log n)$。

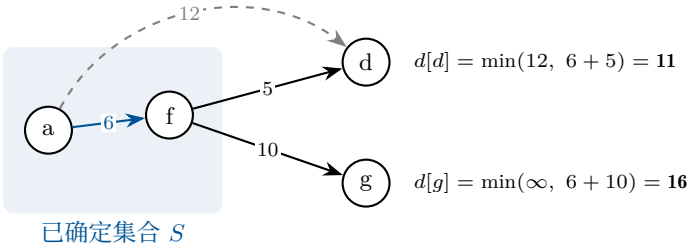



**图 20** 题 7.4 第 3 轮的松弛：f 入 $S$ 后，“经 f 到 d”比直达弧更短，d 被改小

## 6.5 多源最短路径：Floyd 的动态规划

**是什么。**一次求出所有顶点对的距离。
**核心思想。**状态 $D^{(k)}[i][j]$ = “只许经过 $1..k$ 中转”的最短距离；阶段 $k$ 从 0 增到 $n$。这是“逐步放宽约束”式的 DP，和背包问题“逐步放入物品”同构。
**使用条件。**允许负权边，不允许负权回路（若算完后某个 $D[i][i] < 0$，就说明存在负回路）。
**复杂度。**时间 $O(n^3)$，空间 $O(n^2)$。

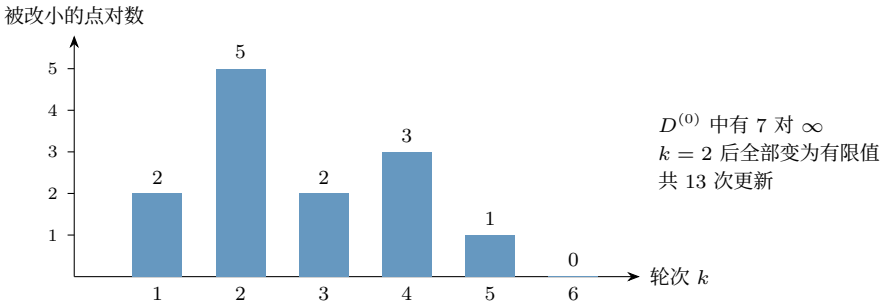



**图 21** 题 7.5 中每一轮被改小的无序点对数（据第二部分的逐轮状态统计）。顶点 2 是连接“左团”$\{1, 3, 4, 5\}$ 与 6 的枢纽，所以 $k = 2$ 时更新最多

## 6.6　相似概念辨析

**表 7**　本章易混概念对比

| 概念组 | 定义要点 | 核心区别 | 常见误区 | 判断方法 |
|---|---|---|---|---|
| **MST vs 最短路径树（图 16）** | MST：总权最小的生成树；SPT：从源点到每点都最短 | MST 无“源点”；SPT 依赖源点 | 认为 MST 上的路径就是最短路径 | 看题目问“总代价”还是“从某点出发” |
| **Prim vs Kruskal（图 6、8）** | 都选安全边 | Prim 长一棵树；Kruskal 合并森林 | 认为二者结果必须相同 | 比较总权；用环性质验证 |
| **Dijkstra vs Floyd vs Bellman–Ford（图 22）** | 单源贪心 / 全源 DP / 单源反复松弛 | 负权：不行 / 可以 / 可以；规模：$O(n^2)$ / $O(n^3)$ / $O(ne)$ | 有负权仍用 Dijkstra | 先看权的符号，再看问“单源”还是“全源” |
| **DFS vs BFS（图 18）** | 栈 / 队列 | BFS 层号 = 最少边数；DFS 无此性质 | 用 DFS 求无权最短路 | 求“最少步数”用 BFS |
| **邻接矩阵 vs 邻接表（图 17）** | $\Theta(n^2)$ / $\Theta(n+e)$ 空间 | 判边快 / 列邻居快 | 有向矩阵当对称 | 稠密用矩阵，稀疏用表 |
| **连通 vs 强连通（图 23）** | 无向图任两点有路 / 有向图任两点互相可达 | 后者要求双向 | 有向图“看起来连成一片”就说强连通 | 检查每对顶点是否双向可达 |

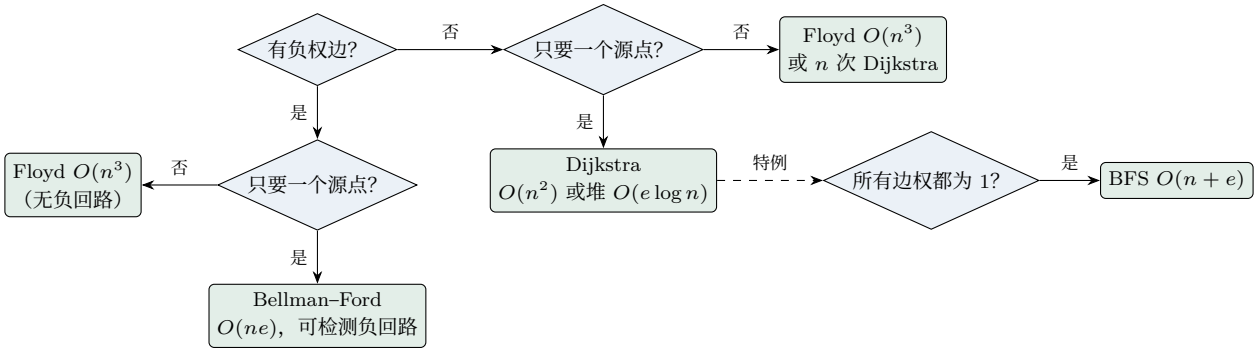
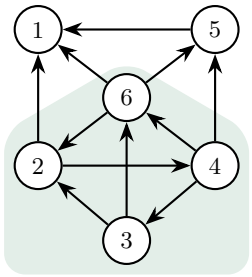



**图 22**　最短路径算法的选择流程（本课程范围内）




题 7.1 的强连通分量：
$\{2,3,4,6\}$（例：2→4→3→2，4→6→2）
{5}、{1} 各自成一个分量
1 没有出弧，任何点都回不来

**图 23**　“连成一片”不等于强连通：题 7.1 的有向图有 3 个强连通分量

## 6.7　知识迁移：这些知识以后在哪里再出现

- **本课程后续**：拓扑排序与关键路径（AOV/AOE 网，本章紧接着的内容）用的是 DFS/BFS 与“在 DAG 上松弛”；并查集是 Kruskal 的核心数据结构，会在“树与等价类”中系统讲授。
- **《算法设计与分析》**：Dijkstra、Prim 是贪心法的范例，Floyd 是动态规划的范例；会进一步学习堆优化、Bellman–Ford、Johnson 算法、A* 搜索、网络流。
- **《计算机网络》**：链路状态路由协议 OSPF 用最短路径树计算路由，距离向量协议 RIP 基于 Bellman–Ford（见第十一章）。

- **控制与机器人**：移动机器人在栅格地图上的全局路径规划、状态机与信号流图本身就是带权有向图（见第九章示例）。

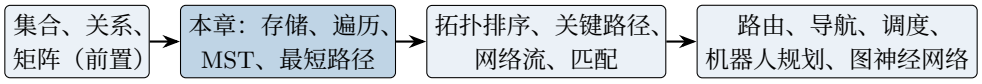
# 第七章　学科科普：图论到底在研究什么

**一句话概括**：图论研究“对象之间的关系”。只要能说清“谁和谁有联系、联系有多强”，问题就能画成一张图：城市与道路、网页与超链接、课程与先修关系、电路节点与导线、蛋白质与相互作用……

**它解决哪些问题。**本章覆盖的三类最具代表性：

- **能不能到**（连通性、可达性）：遍历即可回答，例如“从这台路由器能否到达那个子网”。
- **怎样最省地连通**（最小生成树）：铺设光缆、电网，或在数据分析中按“相似度”把点聚成若干簇。
- **怎样最快地到达**（最短路径）：导航、物流配送、网络路由。

**为什么本科要学。**图是最通用的离散模型，几乎所有后续计算机课程都会把问题“翻译成图”。它在知识链中的位置是：




**理论与现实的关系。**算法课里的“图”是对现实的简化：道路图忽略了红绿灯等待、拥堵随时间变化；电网 MST 忽略了冗余备份要求（真实电网通常**故意**不是树，以防单点故障）。理解模型的假设，和会算模型同样重要。

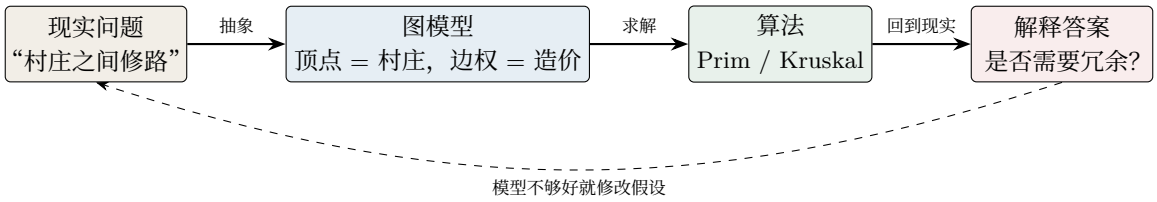



**图 24**　用图论解决问题的完整循环：抽象—求解—回到现实—修正

# 第八章　科学史：从七座桥到导航软件




**图 25**　本章算法的历史时间线（等间距排列、非比例尺；细节与出处见下文与第四部分）

**1735–1741：欧拉与柯尼斯堡七桥——“图”的源头**

柯尼斯堡（今俄罗斯加里宁格勒）有七座桥连接两岸与两座河中陆地，人们想知道能否每座桥恰好走一次。欧拉于 1735 年 8 月 26 日向圣彼得堡科学院报告了解答，论文《关于位置几何的一个问题的解》署 1736 年，实际于 1741 年刊出。他用大写字母表示陆地、小写字母表示桥，把走法写成字母序列，发现关键只在于“与每块陆地相连的桥数是奇是偶”，从而证明这样的走法不存在。

**防止故事化失真**：流行的说法是“欧拉把陆地画成点、桥画成线”，但据数学史研究者的考证，欧拉原文并没有画出今天意义上的“图”，点线图是后人的表述方式；把这篇论文视为图论的开端，是后来的追认。

**与本次作业的联系**：欧拉论证的核心量——“与一块陆地相连的桥数”——就是今天的“度”；题 7.1 求入度、出度并用 $\sum \mathrm{ID} = \sum \mathrm{OD} = e$ 检验，与欧拉“数桥的端点”是同一种计数思想。

**1926–1957：最小生成树被“发现了好几次”**

据 Graham 与 Hell 1985 年发表于 *Annals of the History of Computing* 的史学论文，人们习惯把 MST 问题归功于 Kruskal（1956）与 Prim（1957），但二人的论文其实都引用了更早的 Borůvka（1926）；而且这一问题在捷克斯洛伐克、法国、波兰等地有多个看起来相互独立的来源。

**Borůvka（1926）**在捷克发表了两篇论文，其中一篇题名直译为“对经济地建设电力网络问题的贡献”——当时摩拉维亚地区正在推进电气化，“用最少的线路把各地连起来”是真实的工程需求。

**Jarník（1930）**给出的方法就是今天的“Prim 算法”；Nešetřil 在 1997 年的文章中专门指出，这一事实长期未被广泛知晓。因此一些文献把它称为 Jarník–Prim 算法。

**Kruskal（1956）**在 *Proc. AMS* 发表 *On the shortest spanning subtree of a graph and the traveling salesman problem*；**Prim（1957）**在《贝尔系统技术期刊》发表 *Shortest connection networks and some generalizations*。

**与本次作业的联系**：题 7.2 让我们用两种算法算同一张图，恰好重演了历史——同一问题的两条独立思路，得到同样的最优值。

**1956/1959：Dijkstra 在咖啡馆里的“二十分钟发明”**

Dijkstra 在 2001 年接受 Philip Frana 的访谈（后于 2010 年刊载于 *Communications of the ACM*）时回忆：1956 年他在阿姆斯特丹数学中心工作，需要为新计算机 ARMAC 的公开演示挑选一个“非数学家也能看懂问题和答案”的题目，于是想到“求两座城市之间的最短路线”；他说这个算法是在和未婚妻逛街累了、坐在咖啡馆露台上时想出的，“大约用了二十分钟”，没有用纸笔。论文 *A note on two problems in connexion with graphs* 于 1959 年发表在 *Numerische Mathematik* 第 1 卷第 269–271 页；有趣的是，这篇短文讨论的“两个问题”中另一个正是最小生成树。

**说明**：“二十分钟”“咖啡馆”均来自 Dijkstra 本人数十年后的回忆，属于当事人口述，而非同期记录。

**与本次作业的联系**：题 7.4 中我们手工做的 6 轮“选最小 + 松弛”，就是 1956 年那次演示要让观众看懂的过程。

**1962：Floyd 与 Warshall——同一个三重循环**

1962 年，Floyd 在 *Communications of the ACM* 第 5 卷第 6 期发表了只有一页的 *Algorithm 97: Shortest Path*；同年，Warshall 在 *Journal of the ACM* 第 9 卷发表 *A theorem on Boolean matrices*，用同样的三重循环求有向图的传递闭包（可达关系）。Floyd 的短文开头就引用了 Warshall 的定理。也有文献把该算法称为 Roy–Floyd 或 Floyd–Warshall 算法。

**与本次作业的联系**：把题 7.5 中的“min 与 +”换成“或与且”，同一段代码就从“求最短距离”变成“求谁能到谁”——见第九章的关联示例。

# 第九章　关联学科知识网络与跨课程迁移

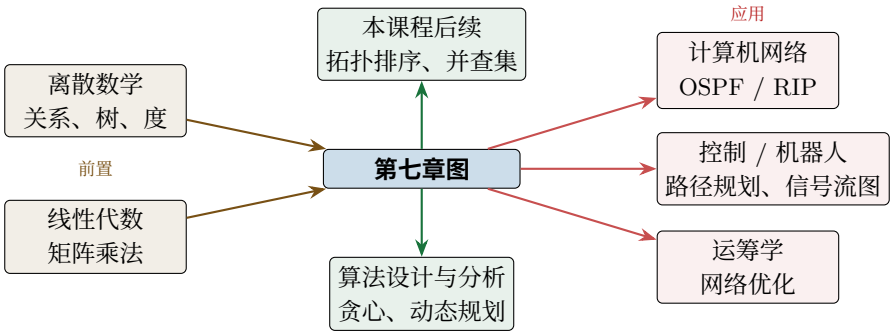



**图 26**　课程知识关系图（只保留最重要的节点）：棕色为前置，绿色为后续，红色为应用

**表 8**　关联知识表（强 = 不懂会直接影响当前学习；拓 = 拓展了解）

| 当前知识点 | 关联课程 | 关联知识点 | 类型 | 为什么有关 | 题号 | 学习层级 |
|---|---|---|---|---|---|---|
| 度、邻接矩阵 | 离散数学 | 关系矩阵、握手定理 | 前置·强 | 邻接矩阵就是“相邻关系”的关系矩阵 | 7.1 | 建议复习 |
| 邻接矩阵 | 线性代数 | 矩阵乘法、$A^k$ | 前置·强 | $(A^k)_{ij}$ 等于长度为 $k$ 的路径条数 | 7.1 | 建议复习 |
| 遍历 | 本课程 | 拓扑排序、关键路径 | 后续·强 | 都在 DFS/BFS 框架上完成 | 7.3 | 必须掌握 |
| Kruskal | 本课程 | 并查集 | 后续·强 | 判断两端是否同属一个分量 | 7.2 | 必须掌握 |
| Dijkstra、Prim | 算法设计 | 贪心法正确性 | 后续·强 | 交换论证、切分性质 | 7.2, 7.4 | 后续系统学习 |
| Floyd | 算法设计 | 动态规划 | 后续·强 | 阶段 $k$、状态 $D[i][j]$ | 7.5 | 后续系统学习 |
| 最短路径 | 计算机网络 | OSPF、RIP | 应用·拓 | 路由表就是最短路径树 | 7.4 | 拓展了解 |
| 最短路径 | 控制 / 机器人 | 栅格地图路径规划 | 应用·拓 | 栅格即图，代价即边权 | 7.4 | 拓展了解 |
| Floyd | 离散数学 | 传递闭包 | 并行·拓 | Warshall 算法 | 7.5 | 拓展了解 |

## 9.1　最小必要知识与小型完整示例

**示例 1（线性代数 · 强关联）：邻接矩阵的平方在数路径**

**背景。**矩阵乘法 $(A^2)_{ij} = \sum_k A[i][k]\,A[k][j]$。当 $A$ 是 0/1 邻接矩阵时，$A[i][k]A[k][j] = 1$ 恰好表示“$i \to k \to j$ 是一条路径”。

**应用于题 7.1。**用第二部分的矩阵 $A$ 计算（程序复核）：

$$A^2 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 2 & 1 & 0 & 1 & 1 & 0 \\ 2 & 2 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}.$$

例如 $(A^2)_{31} = 2$：从 3 出发恰好走 2 步到 1 的路径有 $3 \to 2 \to 1$ 与 $3 \to 6 \to 1$ 两条。第 1、5 行全 0：1 没有出弧；5 只能走到 1，而 1 再走不出去。

**结论。**$(A^k)_{ij}$ 给出长度为 $k$ 的路径条数；把加法换成“或”、乘法换成“且”，累积 $A + A^2 + \cdots + A^n$ 就得到可达矩阵——这正是 Warshall 算法要高效完成的事。

**掌握程度。**会算 $2 \times 2$、$3 \times 3$ 的乘法并解释含义即可；深入可学《离散数学》“关系的闭包”一节。

**示例 2（控制与机器人 · 拓展关联）：栅格地图上的最短路径**

**背景。**移动机器人常把环境划分为栅格：可通行的格子是顶点，上下左右相邻的格子之间连边，障碍格删去。每步代价相同时，最短路径就是 BFS；代价不同（如坡度、地面材质）时用 Dijkstra，工程中常再加上启发式估计，得到 A* 算法（后续课程内容）。

**小例子。**图 27 中 $4 \times 3$ 栅格有 3 个障碍。从 S 做 BFS，各格的层号就是最少步数；目标 G 的层号为 7，沿层号递减回溯即得路径。注意 S 与 G 直线距离只有 3 格，但被障碍挡住后必须绕行。

**掌握程度。**能把栅格画成图并跑 BFS 即可；A*、代价地图属于机器人学 / 人工智能课程。

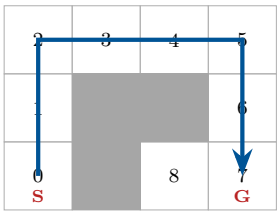



**图 27**　栅格地图就是一张图：BFS 层号即最少步数

# 第十章　国内外高校公开历年试题典型题专题

**检索与收录说明。**本章所有题目均来自可公开访问的页面，访问日期为 2026 年 11 月 2 日。共收录 9 题，来源为剑桥大学（5 题，Computer Science Tripos 官方过往试卷库）、麻省理工学院（2 题，MIT OpenCourseWare 6.006）、加州大学伯克利分校（1 题，CS 170 任课教师主页）以及全国硕士研究生招生考试（1 题）。**如实说明**：国内高校的数据结构期末试卷极少在官方渠道公开，本次未能核验到任何一所国内高校的官方公开试题，因此国内部分仅收录全国统考 408 的一道题，且该题来自多家公开题库的转载（非考试院官方页面）。英文题只做忠实的中文转述或摘要，不复制整份试卷；完整原题请按链接查看。

**【高校公开试题 1】全国硕士研究生招生考试 408（2009）第 41 题　【公开考试真题】**

来源：全国硕士研究生招生考试“计算机学科专业基础综合”，2009 年，综合应用题第 41 题（10 分）。公开来源：新东方在线题库 `https://www.koolearn.com/shiti/dx-st-6151.html`；CSDN 逐题详解 `https://blog.csdn.net/qiwsir/article/details/148626752`（两处题面一致）。对应知识点：Dijkstra 正确性、贪心策略。对应原题：7.4。难度：中等。可信度：多个公开页面交叉核验一致（非官方直接来源）。

**题意（转述）。**非负权带权图中，有人提出如下求“初始顶点到目标顶点最短路径”的方法：路径初始只含初始顶点，令当前顶点 $u$ 为初始顶点；每次选离 $u$ 最近且不在路径中的顶点 $v$ 加入路径，令 $u = v$；直到 $u$ 为目标顶点。问此法能否求得最短路径？能则证明，否则举例。

**考什么、为什么选。**它考的正是第四章“易错 2”的本质：贪心比较的是哪个量。题中方法只看“当前顶点的下一条边”，而 Dijkstra 看“离源点的总距离”。

**解答。不能。**反例（图 28）：边 (1,2)1、(2,3)1、(3,4)1、(1,4)2，求 1 到 4。从 1 出发最近的是 2（1<2），再到 3，再到 4，所得路径 1–2–3–4 长 3；而直达边 1–4 只要 2。另一类反例：贪心可能走进“死胡同”，根本到不了目标。

**检验。**对同一图运行 Dijkstra：1 出队后 dist(2)=1、dist(4)=2；2 出队后 dist(3)=2；4 与 3 并列 2，4 的距离 2 即为正确答案。

**与本次作业的联系。**题 7.4 中 a 的最近邻是 c（2），但到 b 的最短路径恰恰不经过 c——“局部最近”不等于“全局最短”。

**易错点**

**错在哪**：把 Dijkstra 记成“每次走到离当前点最近的点”。**为什么错**：Dijkstra 不是在“走一条路”，而是在“同时维护到所有点的候选距离”。**考场检查**：看算法比较的是 $w(u,v)$ 还是 $d[u]+w(u,v)$。与原作业“易错 2”属同一机制。

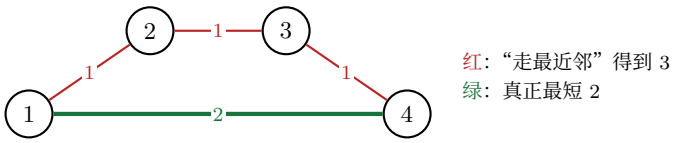



**图 28**　408（2009）第 41 题的最小反例

**【高校公开试题 2】University of Cambridge，Computer Science Tripos 2000，Paper 4，Question 6　【公开真题】**

课程：Data Structures and Algorithms。类型：期末考试题（20 分）。公开来源：`https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2000p4q6.pdf`（剑桥大学计算机系官方试卷库）。对应知识点：Prim、Kruskal、MST 正确性。对应原题：7.2。难度：基础—中等。可信度：官方直接来源。

**题意（转述）。**(a) 详细描述 Prim 与 Kruskal 算法（边权为正的无向图）[7 分]；(b) 解释每个算法为何正确 [7 分]；(c) 比较两者的相对优劣 [6 分]。术语：*minimum cost spanning tree* 最小代价生成树；*explain why … correct* 要求论证而非举例。

**相比本次作业新增的考查角度。**国内作业要求“执行”，这道题要求“描述 + 证明 + 比较”。

**要点答案。**(a) 见第二部分第 7.2 题“解题思路”。(b) 两者都只加入“安全边”：Prim 每步所选边是切分 $(U, V-U)$ 的最小跨越边；Kruskal 所接受的边是两个当前分量之间的最小边（更小的边都已处理过，要么已在森林中、要么成环），也是某切分的最小跨越边。由切分性质及归纳法（“已选边集始终包含于某棵 MST”），两者都得到 MST。(c) Prim：数组实现 $O(n^2)$，不依赖边排序，适合稠密图、邻接矩阵；Kruskal：$O(e \log e)$ 主要花在排序，需并查集，适合稀疏图；若边已排好序，Kruskal 几乎线性。

**易错点。**只写“它们都是贪心所以正确”——贪心本身不保证正确，必须说出切分性质。

**【高校公开试题 3】University of Cambridge，Computer Science Tripos Part IA 2016，Paper 1，Question 10，(b)(d)　【公开真题】**

课程：Algorithms。公开来源：`https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2016p1q10.pdf`。对应知识点：Dijkstra 的适用条件、Bellman–Ford。对应原题：7.4。难度：综合。可信度：官方直接来源。

**题意（转述）。**(b) 讨论 Bellman–Ford 相对 Dijkstra 的优缺点 [4 分]。(d) 若只有**从源点 $s$ 出发的边**允许为负，其余边均非负，Dijkstra 是否仍能求出正确的最短距离？给出证明或反例 [4 分]。

**(b) 要点。**Bellman–Ford 优点：允许负权、能检测负权回路、实现简单；缺点：$O(ne)$，比 Dijkstra 的 $O(e + n \log n)$（斐波那契堆）或 $O(e \log n)$（二叉堆）慢得多。

**(d) 解答：能（前提是图中不存在负权回路，否则最短路径无定义）。**

**证明思路（加常数平移）**：设 $M$ 为足够大的正数，使 $w(s,v) + M \geq 0$ 对所有 $s$ 的出边成立。把 $s$ 的每条出边都加上 $M$，得到全非负的图 $G'$。在没有负回路时，$s$ 到任一顶点的最短路径都是简单路径，恰好用 $s$ 的一条出边，因此在 $G'$ 中每条这样的路径长度都恰好增加 $M$——最短路径不变、所有距离整体加 $M$。Dijkstra 在 $G'$ 上正确；而在原图上，$s$ 第一个出队，此后所有候选距离与 $G'$ 中相差同一个常数 $M$，出队顺序与松弛结果完全对应，所以同样正确。

**检验小例。**$s \to a$ $(-3)$，$s \to b$ $(1)$，$b \to a$ $(1)$：Dijkstra 得 $d(a) = -3$，$d(b) = 1$，与真实值一致。

**与本次作业的联系。**第五章变式 E-2 中负权弧在“中间”就会出错；本题告诉我们：负权出现在“源点出边”这个特殊位置时却无害——判断 Dijkstra 是否可用，关键在于证明中“后续弧非负”这一步是否仍成立。

**【高校公开试题 4】University of Cambridge，Computer Science Tripos Part IA 2017，Paper 1，Question 10，(c)　【公开真题】**

课程：Algorithms。公开来源：`https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2017p1q10.pdf`。对应知识点：DAG 上的最短路径、拓扑序、Dijkstra 复杂度。对应原题：7.3、7.4。难度：综合。可信度：官方直接来源。

**题意（转述）。**(c)(i) 对非负权的有向无环图（DAG），设计 $O(V+E)$ 的算法求从 $s$ 出发的距离，并说明正确性 [4 分]；(ii) 用例子说明为什么 Dijkstra 可能需要 $\Omega(V \log V)$ 时间 [3 分]。（该题 (a)(b) 分别考拓扑排序与 Dijkstra 伪代码及斐波那契堆复杂度。）

**(i) 解答。**先用 DFS（或入度为 0 的队列法）求拓扑序，$O(V+E)$；再按拓扑序依次取每个顶点 $u$，松弛它的全部出边。正确性：DAG 中任一最短路径上的顶点在拓扑序中依次出现，处理 $u$ 时它所有前驱都已处

理完，$d[u]$ 已是最终值，于是沿最短路径逐边归纳即可。每条边只松弛一次，总 $O(V+E)$；且这一方法连负权也能处理。

**(ii) 解答。**取“星形图”：$s$ 到 $v_1,\dots,v_{V-1}$ 各有一条边，权为互不相同的任意数 $x_1,\dots,x_{V-1}$。Dijkstra 会按距离从小到大依次取出 $v_i$，相当于对这 $V-1$ 个数排序；基于比较的排序需要 $\Omega(V\log V)$ 次比较，所以任何基于比较的优先队列实现的 Dijkstra 都至少需要这么多时间——而 (i) 的方法在这张图（也是 DAG）上只需 $O(V)$。

**与本次作业的联系。**题 7.4 的图含回路 b→e→g→b，不是 DAG，所以不能用 (i) 的方法；这提醒我们：先判断图的结构，再选算法。

**【高校公开试题 5】University of Cambridge，Computer Science Tripos Part IA 2018，Paper 1，Question 10，(a)【公开真题】**

课程：Algorithms。公开来源：`https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2018p1q10.pdf`。对应知识点：Dijkstra 的不变式。对应原题：7.4。难度：中等。可信度：官方直接来源。

**题意（转述）。**(a) 证明：若只想求源点到某个目标顶点 $b$ 的最短路径，Dijkstra 可以在 $b$ 被取出（出队）的那一刻停止 [5 分]。((b) 部分在此基础上设计“次短路径”算法并分析复杂度。)

**解答。**Dijkstra 的不变式：每个顶点出队时，其 $d$ 值等于真实最短距离 $\delta(s,\cdot)$（证明见第二部分 7.4 题“为什么正确”）。$b$ 出队时 $d[b]=\delta(s,b)$；此后的迭代只会处理其他顶点，且对已出队顶点不再修改，所以继续运行也不会改变 $d[b]$ 与其前驱链。因此可以立即停止，并沿前驱链输出路径。

**与本次作业的联系。**若题 7.4 只问 a 到 e，表 4 做到第 3 列 e 被选入 $S$ 时就可停止，答案 10 不会再变。

**易错点。**误以为“$b$ 的 $d$ 值第一次变为有限值时”就能停止——那只是候选值（题 7.4 中 g 第一次得到 16，后来改为 14）。

**【高校公开试题 6】University of Cambridge，Computer Science Tripos Part IA 2021，Paper 1，Question 10，(b)【公开真题】**

课程：Algorithms。公开来源：`https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2021p1q10.pdf`。对应知识点：MST 的变形（最大生成树）、Prim 的复杂度。对应原题：7.2。难度：综合。可信度：官方直接来源。

**题意（转述）。**对连通无向带权图，给出求**最大**生成树的 $O(E+V\log V)$ 算法并说明为何满足时间界 [8 分]；(c) 严格证明其正确性 [9 分]；(a) 要求说明“转化策略（translation strategy）”式证明的步骤。

**解答。**把每条边权取相反数 $w'(e)=-w(e)$，在 $w'$ 下求最小生成树：任一生成树 $T$ 有 $w'(T)=-w(T)$，所以 $w'(T)$ 最小 $\iff$ $w(T)$ 最大。求 MST 用**斐波那契堆实现的 Prim**：$V$ 次 extract-min 摊还 $O(\log V)$，至多 $E$ 次 decrease-key 摊还 $O(1)$，合计 $O(E+V\log V)$（等价地，直接在 Prim 中改为“每次选最大边”并用最大堆）。注意 Kruskal 需要排序，一般为 $O(E\log E)$，不满足题目的时间界。

**正确性要点。**切分性质对任意实数权成立（不要求非负），因此取负后 MST 算法仍然正确——这正是“转化策略”：把新问题等价转化为已证明的问题。

**与本次作业的联系。**对题 7.2 的网求最大生成树：同样按 Kruskal 思路从大到小选边，依次接受 (b,e)9、(d,e)7、(d,f)6、(g,h)6、(b,c)5、(c,h)5，最后用 (a,b)4 接入 a，最大总权 $9+7+6+6+5+5+4=42$（程序穷举 3432 种组合验证：最大值为 42，共 5 棵最大生成树，因为权 5 的边之间仍有并列）。

**【高校公开试题 7】MIT 6.006 Introduction to Algorithms，Spring 2020，Quiz 2，Problem 3 “Color Cost” 【公开课程题】**

公开来源：MIT OpenCourseWare，`https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/resources/mit6_006s20_q2_sol/`（含官方解答）。分值 18。对应知识点：图建模、Dijkstra。对应原题：7.4。难度：挑战。可信度：官方直接来源。

**题意（转述）。**无向图边权为正，且每条边被涂成红、绿、蓝之一。一条路径的代价 = 边权之和 + 每次“相邻两条边颜色不同”时额外付出的固定代价 $w_c$。求 $s$ 到 $t$ 的最小代价路径。

**新增的考查角度。**题目代价不只依赖边，还依赖“上一条边的颜色”——普通 Dijkstra 的状态“我在哪个顶点”不够用了，需要**扩展状态**。

**解答（官方思路的中文说明）。**把每个顶点 $v$ 拆成 3 个副本 $(v,\text{红}),(v,\text{绿}),(v,\text{蓝})$，表示“以该颜色的边到达 $v$”。原图中颜色为 $c$ 的边 $(u,v)$ 连接 $(u,c)$ 与 $(v,c)$，权不变；同一顶点不同颜色的副本之间连权为 $w_c$

的边，表示“换色”。新图有 $3V$ 个顶点、$O(V+E)$ 条边，从 $s$ 的各副本出发跑 Dijkstra，取 $t$ 的三个副本中的最小值，时间 $O(E+V\log V)$。

**检验。**若 $w_c=0$，新图中副本之间可以免费切换，答案退化为普通最短路径，符合预期。

**与本次作业的联系。**算法本身仍是题 7.4 的 Dijkstra；难点全部在“把问题翻译成图”。这是国外课程最常见的考法。

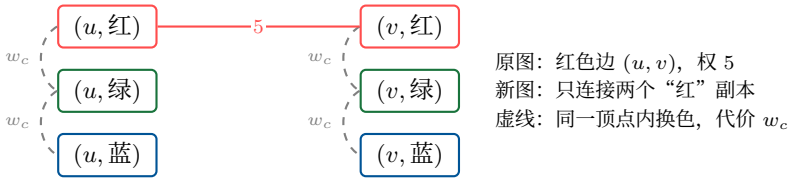



**图 29**　MIT 6.006 “Color Cost”的状态扩展示意：一个顶点拆成三个颜色状态

**【高校公开试题 8】MIT 6.006，Spring 2020，Quiz 2，Problem 4 “Orkscapade”　【公开课程题】**

公开来源同上（含官方解答）。分值 18。对应知识点：BFS、顶点权转化。对应原题：7.3。难度：综合。可信度：官方直接来源。

**题意（转述）。**在一张城镇道路图中，每个城镇驻有若干兽人（Ork），经过一个城镇就要遭遇该镇全部兽人。求从起点到终点、遭遇兽人总数最少的路线，要求 $O(k)$ 时间（$k$ 为兽人总数，可视作远大于城镇数与道路数）。

**解答（官方思路）。**代价在**顶点**上而不在边上。把有 $c$ 个兽人的城镇替换为一条长 $c$ 的顶点链（每个兽人一个顶点），道路连接相应链的端点。于是“路线遭遇的兽人数”等于新图中路径的边数，问题化为无权最短路，用 BFS 在 $O(k)$ 内完成。官方解答特别指出：若改为把权“展开到每条边上”，规模会膨胀到 $O(nk)$。

**与本次作业的联系。**题 7.3 的 BFS 生成树按层展开，层号就是最少弧数；本题告诉我们：只要能把代价“拆成单位长度”，BFS 就是最短路径算法。

**【高校公开试题 9】UC Berkeley CS 170，Fall 2005，Midterm 1，Problem 2 (a)(b)(c)　【公开真题】**

讲师：Michael Jordan。公开来源：`https://people.eecs.berkeley.edu/~jordan/courses/170-fall05/homeworks/midterm1.pdf`（伯克利 EECS 教师个人课程主页）。类型：期中考试“判断并说明理由”，每小题 4 分。对应知识点：MST 与最短路径树的性质。对应原题：7.2、7.4。难度：中等。可信度：官方（课程主页）直接来源。

**题意（转述）。**判断真假并给出理由：(a) 存在无向带权图及顶点 $v$，使最小生成树与以 $v$ 为根的最短路径树没有公共边；(b) 设 $T$ 是 Dijkstra 得到的最短路径树，把每条边权都加上同一个常数 $c>0$ 后，$T$ 仍是最短路径树；(c) 图中权最大的边不可能属于最小生成树。

**(a) 假**（设边权为正）。设 $(v,u)$ 是与 $v$ 相连的最轻边。任一 MST 都含有某条与 $v$ 相连的最轻边（对切分 $\{v\}\,|\,V-\{v\}$ 用切分性质的交换论证）；而 $u$ 到 $v$ 的距离恰为 $w(v,u)$，任何其他路径的第一条边已 $\geq w(v,u)$，再加正权更长，所以最短路径树中 $u$ 的父亲必为 $v$。二者至少共享这条边。

**(b) 假**。反例：$s\to t$ 直达权 3，$s\to a\to t$ 两段各 1。原最短路径为两段（2<3）；每边加 2 后两段为 6、直达为 5，最短路径树改变。原因：加常数对“边数多的路径”惩罚更大。（对比：MST 在每边加同一常数后**不变**，因为所有生成树边数都是 $n-1$。）

**(c) 假**。若最重边是**桥**（删去后图不连通），它必在任何生成树中，例如一条路径图的最重边。题 7.2 中最重边 (b,e)9 不是桥，所以不在 MST 中，但这不是普遍规律。

**易错点。**(b)(c) 都是在用“特殊情形”的直觉替代证明。判断题的标准做法：找到使命题成立所需的条件（“边数相同”“不是桥”），再检查是否总满足。

## 10.1　跨校横向比较与高频母题

**表 9**　同一知识点在不同考试中的考法对比

| 知识点 | 来源 | 题型 / 方法 | 难点 | 易错点 | 原作业 |
|---|---|---|---|---|---|
| MST | 本次作业 | 手算 Prim、Kruskal 状态表 | 等权边 | 以为结果唯一 | 7.2 |
| | 剑桥 2000 | 描述 + 证明 + 比较 | 正确性论证 | 只说“贪心” | 7.2 |
| | 剑桥 2021 | 最大生成树，限定复杂度 | 选对数据结构 | 用 Kruskal 超出时间界 | 7.2 |
| | 伯克利 2005 | 判断并说明理由 | 构造反例 | 以偏概全 | 7.2 |
| Dijkstra | 本次作业 | 手算各步状态 | 绕路更短 | 漏松弛 | 7.4 |
| | 408（2009） | 判断贪心是否正确 | 举反例 | 混淆局部 / 全局 | 7.4 |
| | 剑桥 2016/2018 | 负权条件、提前终止的证明 | 不变式 | 证明不严 | 7.4 |
| | MIT 2020 | 状态扩展建模 | 构图 | 状态设计不完整 | 7.4 |
| BFS | 本次作业 | 画生成树 | 有向矩阵 | 当作对称 | 7.3 |
| | MIT 2020 | 顶点权拆分后 BFS | 化为单位权 | 规模膨胀 | 7.3 |

**表 10**　高频母题结构

| 母题 | 识别信号 | 标准方法 | 常见变化 | 最易错 | 典型来源 |
|---|---|---|---|---|---|
| 最小代价连通 | “连通全部”“总造价” | Prim / Kruskal | 强制含边、最大生成树、唯一性 | 等权、判环 | 7.2；剑桥 2000/2021 |
| 单源最短路 | “从某点到各点” | Dijkstra | 负权、单终点、必经点、提前终止 | 负权、漏松弛 | 7.4；408；剑桥 2016/18 |
| 状态扩展最短路 | 代价依赖“上一步” | 拆点 + Dijkstra/BFS | 颜色、换乘、燃油 | 状态不完整 | MIT 2020 |
| 全源最短路 | “任意两点” | Floyd | 加 / 删边、传递闭包 | 循环顺序 | 7.5 |
| 无权最短路 | “最少步数” | BFS | 栅格、顶点权拆分 | 重复入队 | 7.3；MIT 2020 |

## 10.2　章末考试迁移总结

**国内考法（依据本次作业与 408 真题）**：重视按教材格式手工执行算法并写出状态表，以及“判断方法是否正确并举反例”。

**国外考法（依据检索到的剑桥、MIT、伯克利题目）**：更重视正确性证明（切分性质、不变式）、指定数据结构下的复杂度（斐波那契堆）、把陌生问题建模成图（拆点、反向图、取负转化）以及“判断并说明理由”。

**共同核心**：切分性质、松弛与不变式、DP 阶段——会算还要会说“为什么对”。

**对本次作业的启示**：把题 7.2、7.4 的状态表画熟之后，再试着用一两句话证明每一步为什么安全；遇到新问题先问“顶点是什么、边是什么、代价在哪里”。

**复习优先级**：必须掌握的母题——最小代价连通、单源最短路、全源最短路；必须会辨析——MST vs SPT、Dijkstra vs Bellman–Ford；必须会检查——三角不等式、环性质、度数和；值得挑战——MIT 的 Color Cost 与剑桥 2016(d)。

# 第十一章　生活中的实际应用

本章回答：这些知识藏在我们每天用到的哪些东西里？每个应用都标明是**真实使用**（有公开文档证明该算法在其中使用）还是**类比理解**（原理一致，但具体产品的实现未公开）。自设数值均标“估算值”。

**【生活应用 1】互联网路由：你的数据包怎么找路（真实使用）**

**场景。**打开网页时，数据要经过许多路由器。每台路由器都需要知道“去某个网络，下一跳交给谁”。

**对应知识。**Dijkstra（题 7.4）；Bellman–Ford（第十章试题 3）。

**作用机制。**链路状态协议 OSPF 中，每台路由器收集全网拓扑（链路状态数据库），然后以**自己为根**计算一棵最短路径树；其规范 RFC 2328 写明这一计算使用 Dijkstra 算法（第 16.1 节）。路由表中的“下一跳”就是最短路径树上从根出发的第一条边。另一种协议 RIP 的规范 RFC 2453 写明它基于 Bellman–

Ford（距离向量）算法，并把度量值 16 定义为“无穷大（不可达）”，因此 RIP 网络直径限制在 15 跳以内。

**小型估算。**在图 30 的 5 台路由器网络中，R1 用 Dijkstra 算出到 R5 的最短路径为 R1–R2–R4–R5，代价 $1+2+1=4$（链路代价为估算值），所以去 R5 的下一跳是 R2，而不是直连代价为 6 的 R3 方向。

**模型与现实的差距。**真实网络中链路会故障、代价会被管理员调整，路由器需要反复重算；大型网络还会分区域（area）以减少计算量。

**动手观察（零成本、安全）。**在电脑命令行输入 `tracert www.sdu.edu.cn`（Windows）或 `traceroute`（macOS/Linux），可以看到数据包实际经过的各跳路由器——那就是一条“路由意义下的最短路径”。

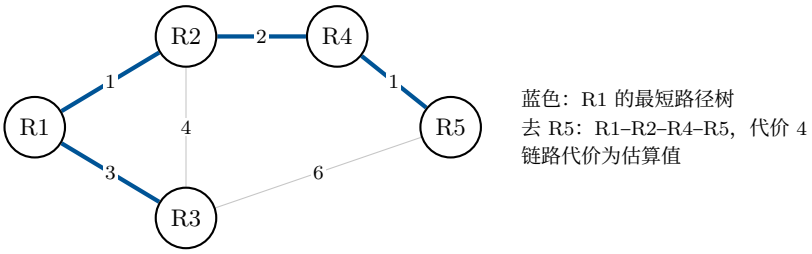



**图 30**　OSPF 路由器以自己为根计算最短路径树（示意，代价为估算值）

**【生活应用 2】手机地图导航（原理真实，具体实现为类比）**

**场景。**在地图 App 中输入“从宿舍到图书馆”，几乎瞬间给出路线和预计时间。

**对应知识。**带权图建模 + Dijkstra（题 7.4）。

**作用机制。**路口为顶点、路段为边、预计通行时间为边权，求一条最短路径。Dijkstra 本人在 2001 年的访谈中就以“车载 GPS 给出从酒店出发的最短路线”来说明他的算法的用途。商业导航软件的具体实现并未公开，业界普遍在 Dijkstra 基础上使用各种加速与预处理技术，并结合实时路况，因此这里把它归为“原理一致的类比”。

**小型估算。**设一座城市道路网有 $n=10^5$ 个路口、$e=3\times10^5$ 条路段（估算值）。教材中的数组版 Dijkstra 约需 $n^2=10^{10}$ 次比较；二叉堆版约需 $(n+e)\log_2 n\approx 4\times10^5\times17\approx7\times10^6$ 次——相差上千倍。这就是为什么“选用什么数据结构实现 Dijkstra”在工程上至关重要。

**模型与现实的差距。**通行时间随时段变化（时变图）；有禁止左转等转向限制（需要像第十章 MIT 题那样扩展状态）。

**【生活应用 3】校园光纤 / 电力线路铺设（MST 的历史原型；数值为估算值）**

**场景。**学校要用光纤把宿舍、食堂、图书馆、教学楼、实验楼连成一张网，各对楼宇间可铺设的线路长度不同，希望总长度最短。

**对应知识。**最小生成树（题 7.2）。第八章提到，Borůvka 1926 年的论文正是研究“经济地建设电力网络”。

**作用机制与计算。**图 31 中（单位：百米，估算值），用 Kruskal：D–S 2、L–J 2 接受；S–L 3、J–E 3 接受，已连通 5 栋楼。MST 总长 $2+2+3+3=10$ 百米；若把 8 条候选线路全部铺设，需 34 百米。

**模型与现实的差距。**真实的电网和校园网通常有意保留冗余环路，以免一条线路故障导致整片断网；MST 给出的是“最省但最脆弱”的方案。

**动手观察。**在校园地图上选 5 栋楼，用步行导航读出两两距离，自己跑一遍 Kruskal。

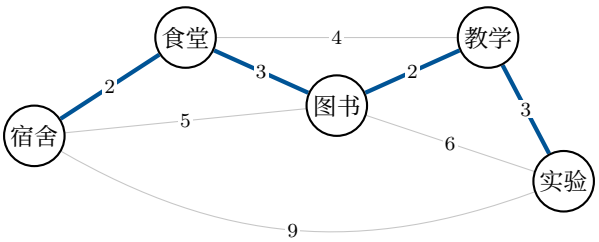



**图 31**　校园光纤铺设的 MST（蓝色，总长 10 百米；所有数值为估算值）

**【生活应用 4】社交软件里的“二度好友”（类比理解）**

**场景。**社交软件提示“你们有 3 位共同好友”或标注“二度人脉”。
**对应知识。**BFS 分层（题 7.3）。
**作用机制。**以你为起点做 BFS：第 1 层是好友，第 2 层是“好友的好友”。层号就是“隔了几度”。各平台的具体实现未公开，这里是原理层面的类比。
**小型估算。**若每人平均有 100 位好友（估算值），第 2 层最多约 $100 \times 100 = 10^4$ 人，第 3 层最多约 $10^6$ 人；由于朋友圈高度重叠，实际人数会少得多——这也是 BFS 必须“入队即标记”、避免重复的现实意义。
**模型与现实的差距。**好友关系有强有弱，“度数”不等于亲密程度。

**【生活应用 5】程序的“垃圾回收”：找出再也用不到的内存（真实使用）**

**场景。**用 Java、Python 等语言编程时，不需要手动释放对象，运行环境会自动回收。
**对应知识。**可达性遍历（题 7.3）。
**作用机制。**把内存中的对象看成顶点、对象之间的引用看成有向弧。追踪式垃圾回收器从“根”（全局变量、栈上变量等）出发做遍历（DFS 或 BFS），所有被访问到的对象被标记为“存活”，没被访问到的就是垃圾，可以回收——这就是经典的“标记—清除”思想。
**小型估算。**若堆中有 $10^6$ 个对象、平均每个对象有 3 个引用（估算值），一次标记约需检查 $10^6 + 3 \times 10^6 = 4 \times 10^6$ 次——与 BFS/DFS 的 $O(n+e)$ 一致。
**模型与现实的差距。**真实回收器还要处理程序在标记过程中继续修改引用等复杂情况，采用分代、并发等技术。

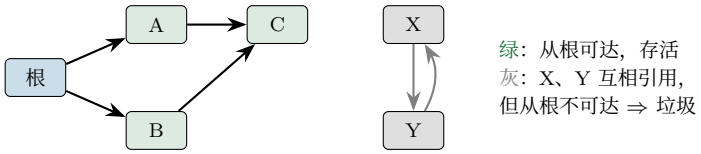



**图 32**　垃圾回收的标记阶段就是一次从“根”出发的遍历；只“互相引用”而不可达的对象也会被回收

**表 11**　知识—生活对照表

| 知识点 | 生活场景 | 起作用的算法 | 类型 | 原题 |
|---|---|---|---|---|
| 单源最短路 | 互联网路由（OSPF / RIP） | Dijkstra / Bellman–Ford | 真实使用 | 7.4 |
| 单源最短路 | 地图导航 | Dijkstra 及其加速版 | 原理真实、实现类比 | 7.4 |
| 最小生成树 | 光纤、电网铺设 | Kruskal / Prim | 真实问题原型（数值估算） | 7.2 |
| BFS 分层 | 社交“几度好友” | BFS | 类比 | 7.3 |
| 可达性遍历 | 垃圾回收标记 | DFS / BFS | 真实使用 | 7.3 |

# 第十二章 学习总结

**本次必须掌握。**(1) 有向图邻接矩阵“行出列入”；(2) DFS/BFS 的执行与生成树；(3) Prim、Kruskal 的状态表与切分性质；(4) Dijkstra 的“选最小 + 松弛”与非负权条件；(5) Floyd 的递推式、$k$ 在最外层、路径还原。

**建议重点复习。**MST 唯一性判断（环性质）；三角不等式检验；同一图上 MST 与最短路径树的区别。

**必须避免的错误。**把有向矩阵当对称；Prim 写成 Dijkstra；有负权仍用 Dijkstra；Floyd 循环顺序写错；BFS 出队才标记。

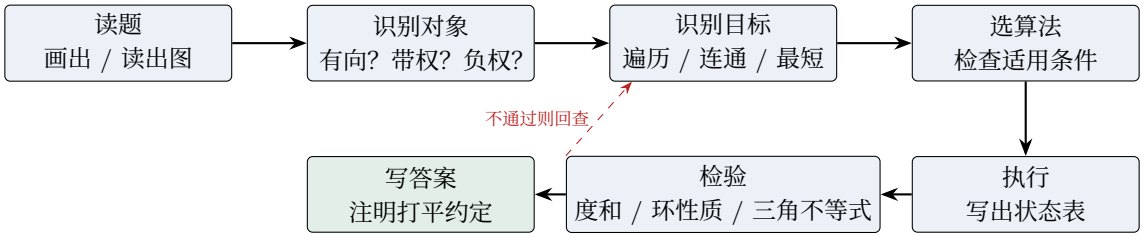



**图 33** 图论题的做题识别流程

**“看到什么 → 想到什么 → 选什么 → 怎么查”思维链：**

| 看到什么条件 | 想到什么知识 | 选择什么方法 | 如何检查答案 |
|---|---|---|---|
| 矩阵不对称 | 有向图 | 只扫行；行和 = 出度 | $\sum \mathrm{ID} = \sum \mathrm{OD} = e$ |
| “最少步数”、边无权 | BFS 层号 | BFS | 树弧两端层号差 1 |
| “连通所有点、总代价最小” | MST、切分性质 | 稠密 Prim，稀疏 Kruskal | 边数 $n-1$；环性质 |
| “从某点到各点”、权 $\geq 0$ | 松弛、贪心 | Dijkstra | 三角不等式；入 $S$ 距离单调 |
| 出现负权边 | Dijkstra 失效 | Bellman–Ford / Floyd | 检查负回路 |
| “任意两点之间” | 动态规划 | Floyd（$k$ 最外层） | 对称性；三角不等式 |
| 代价依赖“上一步状态” | 状态扩展 | 拆点后再求最短路 | 退化情形是否符合预期 |

# 第四部分　参考资料与权威来源

以下只列出本讲义编写过程中实际查阅、用于核验定义、算法、史实、试题与应用的资料。网页的访问日期均为 2026 年 11 月 2 日。

## 一、教材（定义、算法与符号约定）


[1] 严蔚敏，吴伟民.《数据结构（C 语言版）》，清华大学出版社，第 7 章“图”（邻接矩阵 / 邻接表、DFS/BFS、Prim 的 closedge 数组、Dijkstra 状态表格式、Floyd 算法）。

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009. Ch. 22（基本图算法）、Ch. 23（最小生成树：切分性质）、Ch. 24（单源最短路径：松弛、Dijkstra 正确性）、Ch. 25（Floyd–Warshall）。


## 二、原始论文与科学史


[3] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik* 1 (1959), 269–271.

[4] J. B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proc. Amer. Math. Soc.* 7 (1956), 48–50.

[5] R. C. Prim. Shortest connection networks and some generalizations. *Bell System Technical Journal* 36 (1957), 1389–1401.

[6] R. W. Floyd. Algorithm 97: Shortest path. *Communications of the ACM* 5(6) (1962), 345. doi:10.1145/367766.368168.

[7] S. Warshall. A theorem on Boolean matrices. *Journal of the ACM* 9(1) (1962), 11–12. doi:10.1145/321105.321107.

[8] R. L. Graham, P. Hell. On the history of the minimum spanning tree problem. *Annals of the History of Computing* 7(1) (1985), 43–57. doi:10.1109/MAHC.1985.10011. 全文：`https://www.math.u-szeged.hu/~hajnal/courses/MSc_AlgoBony/graham-hell.pdf`

[9] J. Nešetřil. A few remarks on the history of MST-problem. *Archivum Mathematicum* 33 (1997), 15–22.（含 Borůvka 1926 两篇论文与 Jarník 1930 论文的书目信息）`https://dml.cz/handle/10338.dmlcz/107593`

[10] P. L. Frana. An interview with Edsger W. Dijkstra（2001 年访谈，2010 年刊于 *Communications of the ACM*）。`https://cacm.acm.org/news/an-interview-with-edsger-w-dijkstra`

[11] D. Gries. Edsger Dijkstra and the shortest-path algorithm（Cornell CS 2110 课程讲义，引用上述访谈原文）。`https://courses.cs.cornell.edu/cs2110/2017sp/online/shortestPath/01shortestPathHistory.pdf`

[12] T. Paoletti. Leonard Euler's Solution to the Königsberg Bridge Problem. MAA *Convergence* 3 (2006). `https://old.maa.org/node/116597`

[13] H. Kraft. Das Problem der Königsberger Brücken（巴塞尔大学讲义，含欧拉论文 1735 年报告、1741 年刊出及“原文未画图”的史学讨论的引述）。`https://bez.unibas.ch/fileadmin/user_upload/bez/Koenigsberg.pdf`


## 三、高校与考试公开试题（第十章）


[14] University of Cambridge, Department of Computer Science and Technology. Past exam papers: Algorithms（试卷索引）。`https://www.cl.cam.ac.uk/teaching/exams/pastpapers/t-Algorithms.html`

[15] Cambridge CST 2000, Paper 4, Question 6. `https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2000p4q6.pdf`

[16] Cambridge CST Part IA 2016, Paper 1, Question 10. `https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2016p1q10.pdf`

[17] Cambridge CST Part IA 2017, Paper 1, Question 10. `https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2017p1q10.pdf`

[18] Cambridge CST Part IA 2018, Paper 1, Question 10. `https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2018p1q10.pdf`

[19] Cambridge CST Part IA 2021, Paper 1, Question 10. `https://www.cl.cam.ac.uk/teaching/exams/pastpapers/y2021p1q10.pdf`

[20] MIT OpenCourseWare. 6.006 Introduction to Algorithms, Spring 2020, Quizzes 页面与 Quiz 2 Solutions. `https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/pages/quizzes/`; `https://ocw.mi`

t.edu/courses/6-006-introduction-to-algorithms-spring-2020/resources/mit6_006s20_q2_sol/

[21] UC Berkeley CS 170, Fall 2005, Midterm 1（Lecturer: Michael Jordan）. https://people.eecs.berkeley.edu/~jordan/courses/170-fall05/homeworks/midterm1.pdf

[22] 2009 年全国硕士研究生招生考试计算机学科专业基础综合第 41 题（转载页面）：新东方在线 https://www.koolearn.com/shiti/dx-st-6151.html；CSDN “408 考研逐题详解：2009 年第 41 题”https://blog.csdn.net/qiwsir/article/details/148626752

## 四、生活应用的技术文档（第十一章）

[23] J. Moy. RFC 2328: OSPF Version 2, 1998（§1.1、§2.2 以路由器为根的最短路径树；术语表“Shortest-path tree”条目注明由 Dijkstra 算法计算，见 §16.1）。https://www.rfc-editor.org/rfc/rfc2328

[24] G. Malkin. RFC 2453: RIP Version 2, 1998（§3.1 基于 Bellman–Ford（距离向量）算法；度量 16 表示不可达）。https://www.rfc-editor.org/rfc/rfc2453

## 五、计算核验说明

第二、三部分的全部数值结果（度数、Prim/Kruskal 过程、MST 个数与最大生成树、DFS/BFS 序列、Dijkstra 各轮状态、Floyd 各轮矩阵、变式题答案、$A^2$ 等）均由编写者用 Python 独立编程复算；Floyd 逐轮矩阵由程序直接生成为 LaTeX 代码，避免人工抄写错误。